```
F1 Score = 0.818
```

While accuracy is more straightforward, the F_1 score provides more insight into how well your model is doing if you have an imbalanced dataset.

Evaluating a defect classifier on imbalanced data

When dealing with imbalanced datasets—where one class significantly outweighs the other—accuracy alone is often insufficient to measure the effectiveness of a machine learning model. This is particularly true in cases such as defect detection, where the number of defective products (positive class) is far smaller than the number of non-defective products (negative class).

To better understand how imbalanced data can impact model evaluation, we will calculate and compare accuracy, precision, recall, and F_1 score for two defect classification scenarios. In the first scenario, we have few defects and many non-defective examples, while in the second scenario, the data is flipped and defects are more common than non-defective examples.

Scenario 1 – few defects and many non-defective examples

In this case, the model is trained on an imbalanced dataset where defects are rare, and most examples are non-defective. The confusion matrix for the defect classifier is shown here (*Table 11.3*).

Defect classifier	**Prediction (no defect)**	
Actual	No defect	Defect
No defect	18 TNs	0 FPs
Defect	1 FN	1 TP

Table 11.3 – Confusion matrix for a defect classifier with imbalanced data, where most examples are no defect

Let's calculate the accuracy, precision, recall, and F_1 score for a model dealing with imbalanced data (where defects are rare). First, define the true and predicted values for the dataset, then use scikit-learn's metrics to compute these performance measures. Finally, print out the results:

```
y_true_imbalanced = [0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0]
y_pred_imbalanced = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0]
# Calculate performance metrics
accuracy_imbalanced = accuracy_score(y_true_imbalanced,
    y_pred_imbalanced)
precision_imbalanced = precision_score(y_true_imbalanced,
    y_pred_imbalanced)
recall_imbalanced = recall_score(y_true_imbalanced,
    y_pred_imbalanced)
f1_imbalanced = f1_score(y_true_imbalanced,
    y_pred_imbalanced)
print("Accuracy, precision, recall, F1 score for imbalanced classifier")
print(f"Accuracy = {accuracy_imbalanced:.3f}")
print(f"Precision = {precision_imbalanced:.3f}")
print(f"Recall = {recall_imbalanced:.3f}")
print(f"F1 Score = {f1_imbalanced:.3f}")
```

This results in the following output:

```
Accuracy, precision, recall, F1 score for an imbalanced classifier
```

```
Accuracy = 0.950
Precision = 1.000
Recall = 0.500
F1 Score = 0.667
```

In this case, while the accuracy is high at 95%, it is somewhat misleading because the model only detected one out of two defects. The precision of `1.0` means that when the model predicted a defect, it was always correct. However, the recall (`0.5`) shows that the model missed 50% of the actual defects. The F_1 score, which balances precision and recall, is 0.667, indicating there is room for improvement in the model's ability to capture all defects.

Scenario 2 – many defects and few non-defective examples

Now, let's flip the data imbalance and assume most examples are defects, with only a few non-defective cases This scenario is illustrated in *Table 11.4*:

Defect classifier	**Prediction (mostly defects)**	
Actual	No defect	Defect
No defect	1 TN	1 FP
Defect	0 FNs	18 TPs

Table 11.4 – Confusion matrix for a defect classifier with imbalanced data, where most examples are defects

Let's calculate the accuracy, precision, recall, and F_1 score for a model dealing with imbalanced data (mostly defects). First, define the true and predicted values for the dataset, then use scikit-learn's metrics to compute these performance measures. Finally, print out the results:

```
y_true_defects = [1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1]
y_pred_defects = [1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1]
# Calculate performance metrics
accuracy_defects = accuracy_score(y_true_defects,
    y_pred_defects)
precision_defects = precision_score(y_true_defects,
    y_pred_defects)
recall_defects = recall_score(y_true_defects,
    y_pred_defects)
f1_defects = f1_score(y_true_defects, y_pred_defects)
print("Accuracy, precision, recall, F1 score for defect classifier")
print(f"Accuracy = {accuracy_defects:.3f}")
print(f"Precision = {precision_defects:.3f}")
print(f"Recall = {recall_defects:.3f}")
print(f"F1 Score = {f1_defects:.3f}")
```

This gives the following output:

```
Accuracy, precision, recall, F1 score for an imbalanced classifier
Accuracy = 0.950
Precision = 0.947
Recall = 1.000
F1 Score = 0.973
```

Here, the accuracy remains high at 95%, and both the precision (`0.947`) and recall (`1.000`) are also high. The F_1 score is 0.973, indicating that the model is performing well in this scenario. However, the model did miss one of the two non-defective cases, which is important to note. Even though the accuracy, precision, and recall are all high, the model's focus on identifying defects could lead to missing a small number of non-defective cases.

Key takeaways

When evaluating a model trained on imbalanced data, it's important to use precision, recall, and F_1 score, in addition to accuracy. While accuracy may appear high, precision and recall provide more insight into how well the model is identifying positive cases (defects). In cases where defects are rare, high precision but low recall could indicate the model is overly conservative in predicting defects. On the other hand, when defects are more frequent, high recall with slightly lower precision might indicate the model is prone to false positives.

To handle imbalanced data effectively, it's crucial to align the evaluation with the business problem. In most cases, you'll want to ensure that the positive class (defects) is well-represented in the metrics. If missing defects is costly, maximizing recall and F_1 score can be more beneficial than simply optimizing for accuracy.

In the next section, we'll explore graphical assessments of binary classifiers to further analyze performance.

Graphical analysis of binary classification, ROC-AUC curves, and confusion matrix

Using graphs can often help our understanding, so next, we'll make some plots that visualize how well the model is predicting the actual values. The first graph you will make is called the **Receiver Operating Characteristics Curve** (**ROC**). It has an associated metric, the **Area Under the Curve** (**ROC-AUC**), or sometimes just **AUC**. This measures how well a model can differentiate between classes. The higher the AUC, the better the model is at distinguishing between the classifications. Let's plot the graph.

Plot the ROC curve and calculate the AUC using the scikit-learn metrics module. In fact, it will draw a very nice graph and automatically calculate the AUC:

```
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_predictions(y_true, y_pred)
```

This makes the following plot:

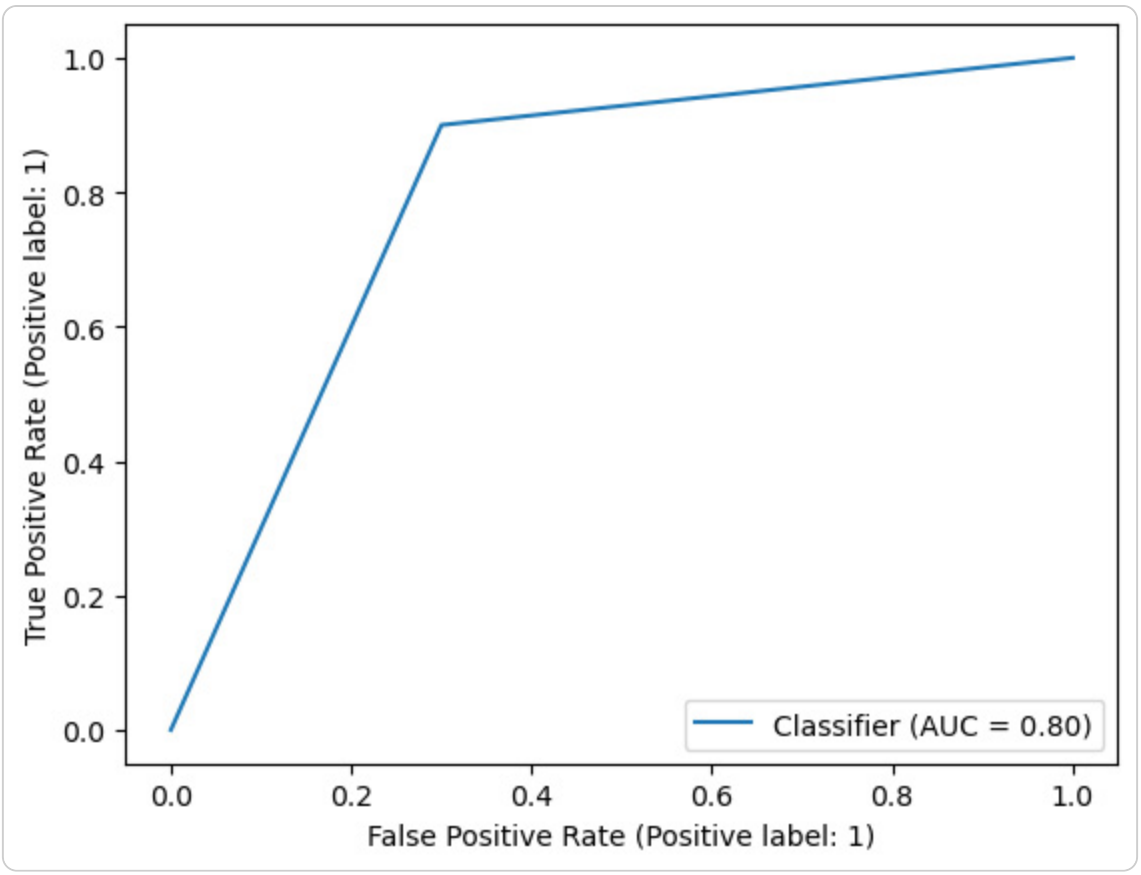


Figure 11.2 – Receiver operating characteristic curve for the defect classifier example

Better models produce a curve closer to the top left corner of the ROC graph, with a higher AUC. A model that randomly associates the samples into classes, a "pure guess," makes a diagonal line through the middle of the graph from bottom left to top right. This defect classifier is doing a good job, with an AUC of 0.8.

The other visual to use is a confusion matrix. This shows how many samples of the data belong to each class compared with where the model classified them:

```
# Generate the confusion matrix
cm = confusion_matrix(y_true_defects, y_pred_defects)
# Create the confusion matrix display
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    display_labels=["No Defect", "Defect"])
# Plot the confusion matrix using the 'Greens' colormap
disp.plot(cmap=plt.cm.Greens)
plt.title("Confusion Matrix for \nDefect Classification")
plt.xlabel("Predicted Class")
plt.ylabel("True Class")
# Add custom text legend to show the 0 → No Defect and 1 → Defect mapping
plt.text(3, 0.1, "Legend:", fontsize=10, weight='bold',
    color='black')
plt.text(3, 0.3, "0 → No Defect", fontsize=9,
    color='black')
plt.text(3, 0.5, "1 → Defect", fontsize=9, color='black')
# Show the plot
plt.tight_layout()
plt.show()
```

This makes the following graph:

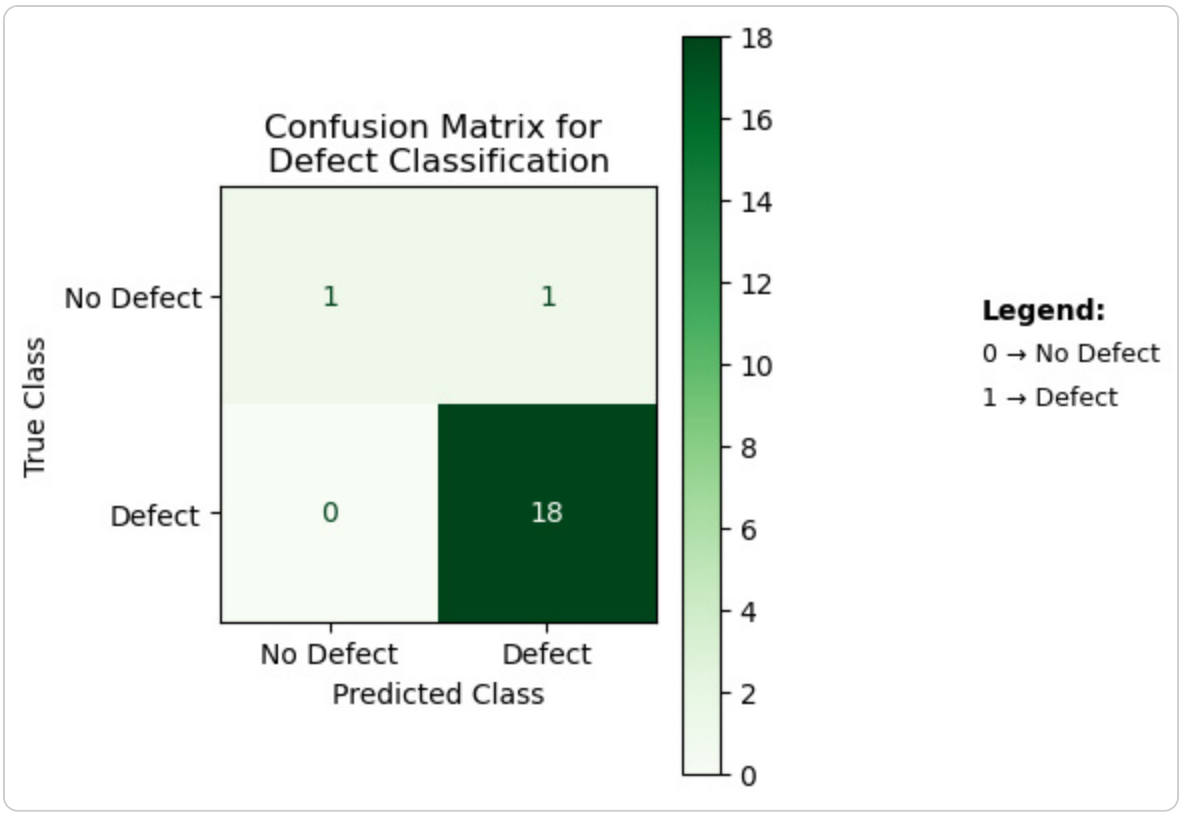


Figure 11.3 – Confusion matrix for the defect classifier example

The visual shows a comparison of predicted classifications to the actual ground truth classifications. The number of TNs, FPs, FNs, and TPs are provided in the matrix. Ideally, the top-right (FPs) and bottom-left (FNs) boxes would contain `0`, indicating perfect classification.

The confusion matrix is particularly useful when you want to understand the breakdown of prediction outcomes across both positive and negative classes. This is especially important when dealing with imbalanced datasets, as it allows you to see whether the model is biased toward predicting one class over the other.

In general, you should use a confusion matrix when you need more insight than a single metric (such as accuracy) can provide. It helps you visualize the types of errors the model is making and can be a better choice for evaluating classification models in cases where class imbalance or specific error types (e.g., FPs) are important.

Next, you'll learn how to calculate classifier metrics when there are more than two classifications.

Multi-class classifier metrics – micro, weighted, and macro averages

If you have more than a binary classification problem, say (for example) you are grading tomatoes and they can be poor, fair, good, or excellent, then there is an additional layer to be added. This layer is how to divide up the problem into a series of binary classification problems. There are two approaches: **One-Versus-One** (**OVO**) or **One-Versus-Rest** (**OVR**). *Table 11.5* shows the groupings for the tomato classification example for both OVO and OVR.

Tomato grading	**Classifiers**
OVO	Poor versus Fair

	Poor versus Good
	Poor versus Excellent
	Fair versus Good
	Fair versus Excellent
	Good versus Excellent
OVR	Poor versus [Fair, Good, Excellent]
	Fair versus [Poor, Good, Excellent]
	Good versus [Poor, Fair, Excellent]
	Excellent versus [Poor, Fair, Good]

Table 11.5 – Table of classifiers for the tomato grading example

As seen in *Table 11.5*, there are only four comparisons that need to be made for the OVR case, compared to six for the OVO case. In general, OVR requires less computational power, so it is more commonly used. It is not perfect, however, because it makes each binary problem imbalanced. By making only one class at a time positive and all the rest of the data negative, you always have a mismatch in the sizes of the classification groups. You can address this by looking at more than just accuracy to evaluate your models, as discussed previously.

Since in the OVR case, we are breaking the problem down into multiple classifiers, we need a way to combine the metrics into a single value for the full model. The **micro average** is the same as accuracy. The **weighted average** takes the class size into account when calculating the average. The **macro average** is the mean of the scores for all of the classifiers. As you saw in the example with mostly defects in the *Binary classifier metrics – accuracy, precision, recall, and F_1 score* section, accuracy can be misleading if classes are imbalanced. For this reason, the micro average is rarely used.

To see how this works, you can use the tomato classification example to explore the metrics for multiclass classifiers:

1. Plot the confusion matrix for our tomato classifier example. You can use the **`ConfusionMatrixDisplay`** function from scikit-learn. Start by setting the true and predicted values for the tomato example. We're pretending a classifier has predicted these results from data:

```
# Define the grades (class labels) and corresponding true/predicted values
grades = ["Excellent", "Good", "Fair", "Poor"]
y_true_tomatoes = [1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3,
    4, 4, 4, 4]
y_pred_tomatoes = [2, 1, 3, 1, 2, 2, 2, 3, 4, 3, 4, 2,
    4, 4, 3, 4]
# Generate the confusion matrix
cm = confusion_matrix(y_true_tomatoes,
    y_pred_tomatoes)
# Create the confusion matrix display with proper class labels
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    display_labels=grades)
# Plot the confusion matrix
disp.plot(cmap=plt.cm.Greens)
```

```
plt.title("Confusion Matrix for \nTomato Grades
    Classification")
plt.xlabel("Predicted Grade")
plt.ylabel("True Grade")
# Add custom text legend to show the grade mapping
plt.text(4.5, 1.5, "Grade Mapping:", fontsize=10,
    weight='bold', color='black')
plt.text(4.5, 1.3, "1 → Excellent", fontsize=9,
    color='black')
plt.text(4.5, 1.1, "2 → Good", fontsize=9,
    color='black')
plt.text(4.5, 0.9, "3 → Fair", fontsize=9,
    color='black')
plt.text(4.5, 0.7, "4 → Poor", fontsize=9,
    color='black')
# Show the plot
plt.tight_layout()
plt.show()
ConfusionMatrixDisplay.from_predictions(
    y_true_tomatoes, y_pred_tomatoes)
```

This prints out the following confusion matrix:

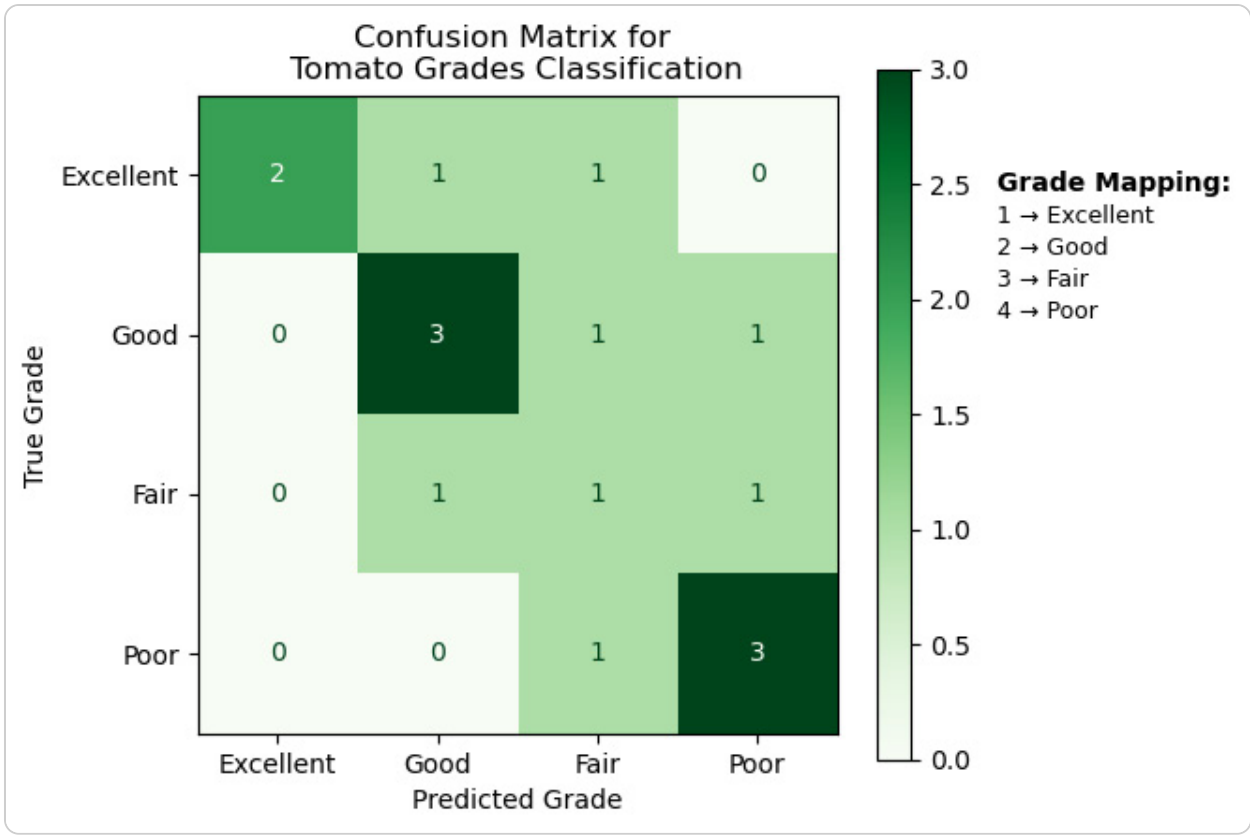


Figure 11.4 – Tomato example confusion matrix

The confusion matrix gives you a visual to see how well each classifier does. In this example, the classifier for **`Poor`** only gets one wrong, while the other three classifiers all get two wrong. Additionally, there are more examples of **`Good`** tomatoes than the other classifications, and the model gets three of the five correct. For **`Fair`** tomatoes, there are only three examples and the model only gets one correct.

2. To print out the classification report from scikit-learn, first, import the **`classification_report`** module, then print it out:

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_true_tomatoes,
    y_pred_tomatoes,target_names=grades))
```

This displays the following output:

```
              precision    recall  f1-score   support

   Excellent       1.00      0.50      0.67         4
        Good       0.60      0.60      0.60         5
        Fair       0.25      0.33      0.29         3
        Poor       0.60      0.75      0.67         4

    accuracy                           0.56        16
   macro avg       0.61      0.55      0.55        16
weighted avg       0.63      0.56      0.57        16
```

Here, you see the precision, recall, and F_1 score for each classifier, as well as how many examples it has (support), which is used in the weighting. As you may have expected, the F_1 score for the classifier that grades **`Poor`** tomatoes and only had one wrong is the best at **`0.67`**. The classifier that grades **`Excellent`** is also high because its precision was high. Each time it said a tomato was **`Excellent`**, it was correct. Now, let's look at how the overall scores are calculated.

The overall accuracy for this OVR multi-class classifier is given by the micro average, dividing the sum of the diagonal cells in the confusion matrix by the sum of all the cells. In this case, that is *(2+3+1+3)/16 = 9/16 = 0.56*.

Then the report gives the weighted and macro average, precision, recall, and F_1 scores. The weighted average is calculated by multiplying the score for the class by the size of the class, then dividing by the total number of samples. The macro average is calculated by taking the arithmetic mean across all the classes. These equations are given in *Table 11.6*. To see how these work, let's do an example using precision:

Precision weighted average = ((excellent score × number of excellent items) + (good score × number of good items) + (fair score × number of fair items) + (poor score × number of poor items)) / total number of items

*= ((1 * 4) + (0.6 * 5) + (0.25 * 3) + (0.6 * 4)) / 16*

= (4 + 3 + 0.75 + 2.4) / 16

= 10.15 / 16

= 0.63

Precision macro average = (excellent score + good score + fair score + poor score)/total number of items

= (1 + 0.6 + 0.25 + 0.6) / 4

= 2.45 / 4

= 0.61

Now you are familiar with how to calculate metrics for when there are multiple classifications. Next, we'll review metrics for regression.

Metrics for regression RMSE R^2

In this section, we will revisit two key metrics used to evaluate regression models: **Root Mean Square Error** (**RMSE**) and **R-squared** (**R^2**). You were first introduced to these metrics in *Chapter 5*, where we used them to compare the performance of XGBoost to other decision tree models. Let's briefly recap what these metrics represent and how they can guide model evaluation:

- **RMSE**: RMSE measures the average difference between the predicted values and the actual values. It provides an indication of how far off the model's predictions are from the true values on average. RMSE values range from 0 to infinity, where lower values indicate better model performance. A lower RMSE means that the model's predictions are closer to the actual outcomes, making it an important metric for assessing accuracy:

$$RMSE = \sqrt{\left(\frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{N}\right)}$$

- **R^2**: R^2 is a statistical measure that represents the proportion of the variance in the target variable that is explained by the model. In other words, it shows how well the model's predictions match the actual data. R^2 values range from 0 to 1, with higher values indicating better performance. An R^2 value of 1 means that the model perfectly explains the variance in the target variable, while a value of 0 means the model explains none of the variance. Sometimes, R^2 is expressed as a percentage; for example, an R^2 of 0.5 could be written as 50%, meaning the model explains 50% of the variance in the target:

$$R^2 = 1\frac{\sum_i (y_i - \hat{y}_i)}{\sum_i (y_i - \bar{y}_i)^2}$$

Now that you understand the key concepts of RMSE and R^2, we'll summarize these metrics in a table below that you can easily refer back to as you evaluate model performance throughout the book.

Evaluating how well a model fits the data summary

To summarize the various metrics used to evaluate model performance, the following table outlines the key metrics and what they tell you about the model's fit. These metrics apply across different model types, such as binary classification, multi-class classification, and regression.

Model type	Fit parameter	Equation	What it tells you
Binary classification	Accuracy	$\frac{(TP+TN)}{(TP+FP+TN+FN)}$	How many of the predictions made by the model were correct
	Precision	$\frac{TP}{(TP+FP)}$	How many of the positive predictions made by the model were correct
	Recall	$\frac{TP}{(TP+FN)}$	How many correct positive predictions were made by the model as a fraction of all the positive cases
	F_1 Score	$2 \cdot \frac{(precision \cdot recall)}{(precision + recall)}$	Combination of precision and recall into a single metric
	ROC AUC	Area under the ROC curve	How well the model differentiates between classes
Multi-class classification	Macro precision	$\frac{\sum_{i=1}^{number\ of\ classes} precision_i}{number\ of\ classes}$	Arithmetic mean of all the precision scores
	Weighted precision	$\frac{\sum_{i=1}^{number\ of\ classes} (precision_i)(class\ size_i)}{total\ samples}$	Weighted average of class-wise precision values weighted by class size
	Macro recall	$\frac{\sum_{i=1}^{number\ of\ classes} recall_i}{number\ of\ classes}$	Arithmetic mean of all class-wise recall values
	Weighted recall	$\frac{\sum_{i=1}^{number\ of\ classes} (recall_i)(class\ size_i)}{total\ samples}$	Weighted average of class-wise recall values weighted by class size
	Macro F_1 score	$\frac{\sum_{i=1}^{number\ of\ classes} F_{1_i}}{number\ of\ classes}$	Arithmetic mean of all class-wise F_1 score values

Model type	Fit parameter	Equation	What it tells you
	Weighted F_1 score	$\frac{\sum_{i=1}^{number\ of\ classes}(F_1)(class\ size_i)}{total\ samples}$	Weighted average of class-wise F_1 score values weighted by class size
	Macro ROC AUC	$\frac{\sum_{i=1}^{number\ of\ classes} ROC\ AUC_i}{number\ of\ classes}$	Arithmetic mean of all class-wise ROC AUC score values
	Weighted ROC AUC	$\frac{\sum_{i=1}^{number\ of\ classes}(ROC\ AUC_i)(class\ size_i)}{total\ samples}$	Weighted average of class-wise ROC AUC score values weighted by class size
Regression	RMSE	$RMSE = \sqrt{\left(\frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{N}\right)}$	Measures the difference between the model and the actual values
	R^2	$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_i)^2}$	How much of the variance of the target is predicted by the model

Table 11.6 – Summary of metrics to measure model fit

In addition to the metrics for how well the model fits the data, you will also want to look at performance metrics to assess how well the model is doing and whether you want to adjust hyperparameter settings. Next, you'll learn about measuring training and inference time.

Model performance, training, and inference time

In addition to measuring how well your model fits the data, you also want to ensure that you are producing an efficient model in terms of the required training and inference time. Based on the need for precision, recall, and F_1 score, you may choose a model that takes a bit longer to train because it provides a better result. On the other hand, you may find that you are in a situation where there are diminishing returns, and a model that requires more training time produces only a small improvement in the performance metrics.

To measure the training and inference time required, you can use **`%%time`** within a Jupyter notebook as you did in *Chapter 5*. In the Python code, you can use the **`time.perf_counter()`** function to measure the execution time of a section of code. In this next example, you'll measure the time to count to 10 using a **`for`** loop. To measure training time, replace the **`for`** loop with your code to train the model. To measure inference time, replace the **`for`** loop with your code to make a prediction.

Measure the time it takes to run a section of Python code. You can use the **`time.perf_counter()`** function. Before the block of code to measure, store the start time by calling **`start_time = time.perf_counter()`**. After the block of code to measure, store the end time by calling **`end_time = time.perf_counter()`**. The time it takes for the code to run is given by the difference between the end and start values. The **`time.perf_counter()`** function measures elapsed time in seconds:

```
import time
start_time = time.perf_counter()
# replace this for loop with what you want to measure
for i in range(10):
    print(i)
end_time = time.perf_counter()
elapsed_time = end_time - start_time
print (f"The code executed in {elapsed_time:.5f} seconds")
```

This results in the following output:

```
0
1
```

```
2
3
4
5
6
7
8
9
The code executed in 0.00025 seconds
```

You now have all the measurements you need to compare models in terms of their performance (model fit metrics) and efficiency (training and inference time). Next, you will learn about how to tune the parameters in XGBoost to improve these metrics.

Tuning XGBoost hyperparameters to improve model fit and efficiency

In the previous section, we discussed various metrics to measure how well a model fits the dataset. Now, we will explore how to optimize an XGBoost model by tuning its hyperparameters to improve both accuracy and efficiency. Hyperparameter tuning is crucial in extracting the best performance from XGBoost, as it helps balance model complexity and overfitting, while also improving computation efficiency.

What is hyperparameter tuning?

Hyperparameter tuning is an optimization process that involves adjusting parameters that control the training process of the model. These parameters are not learned from the data but are set before the training process begins. By fine-tuning them, you can significantly improve the model's performance.

There are several methods you can use to systematically tune these hyperparameters:

- **Grid search**: This method tries every combination of hyperparameters in a specified range. It is computationally expensive but guarantees finding the best combination within the search space.
- **Random search**: This method samples a random subset of the hyperparameter space, making it faster than grid search, though it may not always find the optimal solution.
- **Bayesian optimization**: This method builds a probabilistic model of the objective function and selects the most promising hyperparameters to try. It is more efficient than grid search and random search for large search spaces.

Before diving into the tuning process, it's important to select the right evaluation metric based on the task at hand. Here are some examples:

- For regression models, RMSE or R^2 might be appropriate
- For binary classification, AUC or F_1 score could be better suited

As you tune hyperparameters, you should evaluate model performance using a training dataset and then check performance on a test dataset to avoid overfitting. Let's now take a deeper dive into the specific hyperparameters that you can tune in XGBoost.

Key XGBoost hyperparameters

Here are some of the key XGBoost hyperparameters:

- **Learning rate** (`learning_rate`): The learning rate controls how much the model adjusts with each update. A smaller learning rate makes the model more cautious but requires more boosting rounds (trees) to converge.

Tuning tip: Start with a default value of `0.1` and adjust it down (e.g., to `0.01` or `0.001`) if overfitting occurs. Lower learning rates often require increasing the number of trees (`n_estimators`) to maintain accuracy.

- **Number of trees** (`n_estimators`): This parameter specifies the number of boosting rounds (or trees). Too many trees can cause overfitting, while too few might result in underfitting.

  Tuning tip: Start with a modest number (e.g., 100) and increase it gradually while watching for signs of overfitting. You can compensate for too many trees by decreasing the learning rate.

- **Maximum depth of trees** (`max_depth`): This controls the depth of each individual tree. Deeper trees allow the model to learn more complex patterns, but they also increase the risk of overfitting.

  Tuning tip: Typically, values between 3 and 10 work well, but this varies with the dataset. Shallow trees (e.g., `max_depth = 3`) prevent overfitting but might underfit, while deeper trees (e.g., `max_depth = 10`) capture more complexity.

- **Minimum child weight** (`min_child_weight`): This parameter sets the minimum sum of instance weights (Hessian) needed in a child node. It controls the model's sensitivity to outliers by preventing the model from learning overly specific patterns.

  Tuning tip: Increase `min_child_weight` to prevent overfitting if your model performs poorly on validation data. Default values range from 1 to 6, depending on the dataset size.

- **Gamma** (`gamma`): Gamma specifies the minimum loss reduction required to make a split. A higher gamma value makes the algorithm more conservative in splitting nodes, reducing overfitting.

  Tuning Tip: Start with the default of 0 and gradually increase (e.g., to 0.1 or 0.2) if you notice the model is overfitting. Higher values might reduce model complexity.

- **Subsample** (`subsample`): This parameter controls the proportion of training data used for each boosting round. Using less than 1.0 can help prevent overfitting.

  Tuning tip: A common range for subsample is between 0.5 and 1.0. Lowering this value (e.g., to 0.7) forces the model to use only a portion of the data in each round, reducing variance but potentially increasing bias.

- **Columns to be sampled** (`colsample_bytree`): This parameter controls the fraction of features (columns) to be randomly sampled for each tree. Reducing the number of features per tree can help prevent overfitting.

  Tuning tip: Typically, values between 0.5 and 1.0 are used. Lower values reduce overfitting but may miss important relationships between features.

- **Regularization parameters** (`lambda` and `alpha`): These parameters control L2 (ridge) and L1 (lasso) regularization, respectively. They add a penalty for larger weights, which helps control model complexity.

  Tuning tip: Increasing `lambda` (L2) or `alpha` (L1) can reduce overfitting. Start with `lambda = 1` and `alpha = 0` and increase as necessary based on cross-validation performance.

Now that we know the key hyperparameters, let's look at how to tune the model.

Practical steps for tuning

Here's a step-by-step approach to tuning XGBoost hyperparameters:

1. **Start with `learning_rate`**: Set a value (e.g., 0.1), and later refine it if the model starts overfitting. Small learning rates require more boosting rounds.
2. **Tune `n_estimators`**: Begin with a small number of trees (e.g., 100), then increase based on performance, adjusting `learning_rate` accordingly.

3. **Adjust tree complexity**: Tune `max_depth`, `min_child_weight`, and `gamma` to control the complexity of each tree. Start with moderate values (e.g., `max_depth = 6`, `min_child_weight = 1`, and `gamma = 0`) and increase or decrease as necessary.
4. **Refine subsampling parameters**: Fine-tune `subsample` and `colsample_bytree` to prevent overfitting. Use values that are less than 1.0 to introduce randomness into the training process.
5. **Regularization**: Adjust `lambda` and `alpha` to control model complexity and prevent overfitting.
6. **Iterate and refine**: Once the core parameters are optimized, go back and fine-tune `learning_rate` again if necessary to balance convergence speed and accuracy.

Hyperparameter tuning is essential to maximizing the performance of your XGBoost models. The settings of these hyperparameters depend heavily on the specific dataset and problem you are solving. By following a structured tuning approach, using a combination of search techniques, and regularly validating your model, you can achieve a well-balanced model that performs well without overfitting.

Summary

In this chapter, we explored the critical steps of evaluating a model's performance and fine-tuning its parameters to enhance accuracy and efficiency. You gained practical experience with both the scikit-learn and XGBoost Python APIs, applying key performance metrics to classification and regression models.

We delved into hyperparameter tuning, learning how to adjust parameters such as learning rate, number of trees, and regularization factors to improve model fit while avoiding overfitting or underfitting. By closely monitoring performance on both training and test datasets, you ensured that your model generalizes well beyond the initial dataset.

In the next chapter, we'll shift focus to *managing the feature engineering pipeline*, a vital component in maintaining consistency and efficiency when deploying models in production environments.

12
Managing a Feature Engineering Pipeline in Training and Inference

Feature engineering is one of the most pivotal steps in any machine learning project, as it transforms raw data into a structured form that can be effectively used by machine learning models. This process involves creating new features, selecting relevant ones, and manipulating data in a way that enhances the predictive power of the model. The quality of feature engineering can make or break a model's performance, and it often requires a deep understanding of both the data and the domain.

In production environments, *consistency* between feature engineering during the training and inference stages is critical. Any misalignment in how features are engineered across these two stages can lead to significant performance degradation. For instance, if the model is trained with one set of transformations or feature logic but a different approach is applied when making predictions, the model's accuracy, reliability, and overall performance will suffer. Therefore, ensuring that feature engineering is seamlessly integrated into both training and inference pipelines is essential for a robust deployment.

In this chapter, you will delve into the intricacies of managing feature engineering pipelines specifically tailored for **time series forecasting** and **regression tasks** using the **XGBoost model**, expanding on the concepts and code examples discussed in *Chapters 7* and *9*.

You will begin by exploring strategies to streamline the feature engineering process. This involves automating repetitive tasks, selecting appropriate transformations, and ensuring that these transformations can be efficiently applied to large datasets. A special focus will be placed on creating reusable, well-structured pipelines that can be deployed across different stages of a project—from initial model training to real-time inference. You will explore tools such as **scikit-learn's Pipeline**, custom functions, and utility scripts, demonstrating how to build robust, end-to-end workflows that simplify deployment. By designing modular and reusable components, we can avoid redundancy and potential errors while ensuring that feature engineering logic is not duplicated or altered across environments.

In this chapter, we will cover the following topics:

- Using pipelines to avoid data leaks
- Introducing scikit-learn's Pipeline
- Implementing a data pipeline in Python

Technical requirements

The code presented in this chapter is available in our GitHub repository: https://github.com/PacktPublishing/XGBoost-for-Regression-Predictive-Modeling-and-Time-Series-Analysis/blob/main/ch12/feature_engineering_pipeline_train_inference.ipynb

As usual, you will need to install the software and Python packages in the following list in order to follow along with the chapter:

- Python 3.9 (a virtual environment is recommended)
- pandas 1.4.2 or 2.1.4
- Jupyter Notebook or JupyterLab
- scikit-learn 1.4.2
- XGBoost 2.0.3

Using pipelines to avoid data leaks

One of the most common issues when working with machine learning models, particularly with time series data, is the potential for **data leakage**. Data leakage occurs when information from outside the training dataset is inadvertently used to train the model, often leading to over-optimistic performance metrics. Pipelines can play a vital role in mitigating this risk by ensuring that the same preprocessing steps applied during training are also applied consistently during inference.

For example, when working with time series data, applying transformations such as scaling or encoding to the entire dataset before splitting into training and testing sets can cause future information to leak into the training process. By using a pipeline, you can avoid this issue by ensuring transformations are applied only to the training data during training and then to the test data during inference.

Introducing scikit-learn's Pipeline

Scikit-learn's Pipeline module provides a powerful way to streamline machine learning workflows by chaining data transformation and model training steps into a single, unified process. Instead of manually tracking and applying each preprocessing step, you can package them together into a pipeline, ensuring that all steps are applied consistently and in the correct order.

One of the major advantages of using a pipeline is that it integrates seamlessly with scikit-learn's other tools, such as cross-validation, grid search, and feature selection. This helps to eliminate errors and ensures that the same transformations are applied across both the training and testing stages.

In the following sections, we'll discuss how to manage feature engineering pipelines specifically for time series forecasting and regression tasks using the XGBoost model. Time series data introduces unique challenges—such as the temporal dependence between observations and the necessity to create lagged features, rolling statistics, and other time-based transformations—that require careful attention. On the other hand, regression tasks can vary in complexity depending on the nature of the data and the relationships between features.

Implementing a data pipeline in Python

Now that we've discussed the importance of consistency in feature engineering across training and inference, it's time to dive into the practical side of building data pipelines. In this chapter, we will focus on implementing feature engineering pipelines in Python that will allow you to automate the process, making it repeatable and reliable for both time series forecasting and regression tasks. This reduces the risk of data leakage and ensures that your model performs optimally when it's deployed in production. Let's explore how to implement these pipelines using Python's scikit-learn and XGBoost libraries.

Time series feature engineering in a pipeline

As you learned in *Chapter 9*, time series data poses unique challenges due to the temporal dependencies between observations. In this section, you will perform feature engineering for time series data and use a pipeline to combine the features with model training. Using a pipeline allows you to perform the same tasks again and again on training and test data.

Using a pipeline with time series data

One of the most common feature engineering techniques for time series is the creation of **lagged features**, which allow the model to capture trends and dependencies over time. In this section, you'll practice feature engineering and use a pipeline to fit an XGBoost model and make predictions. Let's get started:

1. First, import the necessary libraries. You have used most of these in previous chapters. A new library is `Pipeline`:

```
import pandas as pd
import numpy as np
from xgboost import XGBRegressor
from sklearn.pipeline import Pipeline
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.impute import SimpleImputer
from sklearn.base import BaseEstimator, TransformerMixin
```

2. Next, create a custom transformer to make lagged features. This uses the concepts introduced in *Chapter 9*. Here, you will create a class called **`LagFeatureTransformer`** to make lagged features by shifting the values of the time series by a specified number of lags. These lagged values are then used as features in the model:

```
# Custom transformer for creating lagged features
class LagFeatureTransformer(BaseEstimator,
    TransformerMixin):
    def __init__(self, lags=3):
        self.lags = lags
    def fit(self, X, y=None):
        return self
    def transform(self, X, y=None):
        df = pd.DataFrame(X.copy())
        for lag in range(1, self.lags + 1):
            df[f'lag_{lag}'] = df['Value'].shift(lag)
            df.dropna(inplace=True)
        return df
```

3. Prepare the time series dataset by creating a synthetic time series dataset and apply **`LagFeatureTransformer`** to generate lagged features for training a regression model:

```
date_range = pd.date_range(start='1/1/2020',
    periods=100, freq='D')
data = pd.DataFrame({'Date': date_range,
    'Value': np.random.randn(100).cumsum()})
data.set_index('Date', inplace=True)
lagged_data = LagFeatureTransformer(
    lags=3).transform(data)
X = lagged_data.drop(columns=['Value'])
# Features: lagged values
y = lagged_data['Value']
# Target: original values shifted by lag
```

4. Now that you have created the lagged features, you are ready to use them inside a scikit-learn pipeline that includes both feature scaling and model training using XGBoost. The pipeline allows for streamlined preprocessing and model training, ensuring that the same steps are applied consistently across both the training and inference phases:

```
X_train, X_test, y_train, y_test = train_test_split(X,
    y, test_size=0.2, shuffle=False, random_state=42)
# Define the pipeline with scaling and XGBoost model
pipeline = Pipeline(steps=[
    ('scaler', StandardScaler()),  # Feature scaling
    ('model',XGBRegressor(
        objective='reg:squarederror'))
    # XGBoost regression model
])
# Train the pipeline
pipeline.fit(X_train, y_train)
# Make predictions
y_pred = pipeline.predict(X_test)
```

The pipeline you created in this section performs feature scaling and specifies the model type and hyperparameters you wish to use. The steps in the pipeline are performed in the order you set each time the pipeline is called for either model fitting or model prediction tasks. This allows you to repeat a flow of steps (here, just two steps, and you can see how you could expand it to many steps) every time the model is trained or a prediction is made.

Evaluating the time series model

After training, you can evaluate the model's performance using standard regression metrics such as **mean squared error** (**MSE**) and R^2. Let's do that now:

1. Evaluate the model using MSE and R^2 metrics. These metrics are built into scikit-learn in the `metrics` module:

```
mse = mean_squared_error(y_test, y_pred)
r_squared = r2_score(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
print(f'R squared: {r_squared}')
```

This outputs the following:

```
Mean Squared Error: 5.384843571846216
R squared: -2.35336221354622
               Actual  Predicted
Date
2020-03-21 -19.313636 -20.451332
2020-03-22 -19.821886 -20.661566
2020-03-23 -18.626756 -21.220581
2020-03-24 -18.911709 -20.336151
2020-03-25 -18.478816 -20.611525
```

In this section, you created a pipeline to combine multiple steps for preparing and executing model training for time series data. This pipeline handles feature engineering and model training in a unified process, making it easier to deploy and manage models. Next, you will build a pipeline for a regression model.

Regression feature engineering in a pipeline

As you learned in *Chapter 7*, regression tasks typically involve datasets with numerical and categorical features. Managing missing values and standardizing features are necessary for building effective regression models. Frequently, a number of feature engineering tasks are necessary to prepare a dataset for modeling. Having all those tasks grouped together in a pipeline makes the data preparation process more repeatable. Next, you will perform all the same feature engineering steps you did in *Chapter 7* in a pipeline.

Using a pipeline with regression data

Real-world datasets often contain missing values in both numeric and categorical variables. In this section, you will build a pipeline that performs the feature engineering tasks needed to prepare the housing dataset from *Chapter 7* for modeling, train a model with XGBoost, and perform inference.

Handling missing values in categorical data

To handle missing values in categorical data, you can apply different strategies depending on the proportion of missing entries. In this example, we will use a custom transformer called `MissingValueImputer`, which either replaces missing values with the string `Missing` for variables with a high proportion of missing values or imputes the most frequent category for variables with fewer missing entries. This ensures that the dataset is complete before being fed into the model.

`BaseEstimator` and `TransformerMixin` from scikit-learn are essential for creating reusable and consistent transformers that integrate seamlessly into scikit-learn's ecosystem. These classes allow us to create custom transformers that can be incorporated into pipelines. `BaseEstimator` provides methods such as `get_params` and `set_params`, making your custom transformer compatible with pipelines and hyperparameter tuning, while `TransformerMixin` adds the `fit_transform` method, ensuring the transformer can be used in both training and inference stages.

Here's the custom transformer:

```
class MissingValueImputer(BaseEstimator, TransformerMixin):
    def __init__(
    self, for_missing_string,for_frequent_category):
        self.for_missing_string = for_missing_string
        self.for_frequent_category = for_frequent_category
        self.frequent_categories = {}
    def fit(self, X, y=None):
        # Store the most frequent category for variables
        #with few missing observations
            for var in self.for_frequent_category:
                self.frequent_categories[var] =X[
                    var].mode()[0]
            return self
    def transform(self, X, y=None):
        # Replace missing values with "Missing" for
        #specific variables
        X[self.for_missing_string] = X[
            self.for_missing_string].fillna('Missing')
           # Replace missing values with the most frequent
           #category for specific variables
            for var in self.for_frequent_category:
                X[var] = X[var].fillna(
                    self.frequent_categories[var])
            return X
```

This transformer ensures that categorical variables are properly preprocessed before model training, either by imputing missing values with the most frequent category or replacing them with the string **`Missing`**.

By inheriting from **`BaseEstimator`** and **`TransformerMixin`**, your transformer becomes reusable and compatible with scikit-learn's entire pipeline framework, enabling efficient, consistent transformations during both training and inference.

Preparing the dataset

Next, we will prepare the dataset for modeling. We use the housing dataset from *Chapter 7*, which contains both numeric and categorical features. The target variable is **`SalePrice`**, and we will preprocess both feature types accordingly:

```
data = pd.read_csv("house_pricing.csv")
# Define feature columns
numeric_features = ['LotArea', 'YearBuilt', '1stFlrSF',
    '2ndFlrSF', 'GrLivArea']
categorical_features = ['Neighborhood', 'HouseStyle',
    'GarageType', 'SaleCondition']
# Handle missing categorical variables
cat_vars_with_na = [var for var in categorical_features if data[var].isnull().sum() > 0]
for_missing_string = [var for var in cat_vars_with_na if data[var].isnull().mean() > 0.1]
for_frequent_category = [var for var in cat_vars_with_na if data[var].isnull().mean() <
0.1]
# Target column
target = 'SalePrice'
```

Next, split the data into features and **`target`** and perform the **`train`**/**`test`** split:

```
X = data.drop(columns=[target, 'Id'])
y = data[target]
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)
```

Now the data is prepared, you can move on to building the pipeline.

Building the pipeline

Now that we've prepared the dataset, we can build the full pipeline that combines both numeric and categorical feature preprocessing with model training using XGBoost:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
     # Handle missing values
    ('scaler', StandardScaler())     # Scale features
])
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    # Handle missing categorical values
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
     # One-hot encode categorical features
])
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer,
             categorical_features)
    ])
pipeline = Pipeline(steps=[
    ('missing_imputer', MissingValueImputer(
        for_missing_string, for_frequent_category)),
    # Custom missing value handling
    ('preprocessor', preprocessor),
    # Preprocessing step
    ('model', XGBRegressor(objective='reg:squarederror'))
    # XGBoost regression model
])
```

Evaluating the model

Now that the pipeline is set up, you can train and evaluate the regression model using the MSE and R^2 metrics:

```
# Train the pipeline
pipeline.fit(X_train, y_train)
# Make predictions
y_pred = pipeline.predict(X_test)
# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
# Display the predicted vs actual values
result = pd.DataFrame({'Actual': y_test,
    'Predicted': y_pred}, index=X_test.index)
print(result.head())
```

This prints the following output:

```
Mean Squared Error: 1130202899.0655065
      Actual      Predicted
892   154500  142317.156250
1105  325000  364155.031250
413   115000   93371.398438
522   159000  135780.375000
1036  315500  266444.375000
```

In this section, we combined data preparation through feature engineering and model training using pipelines. By using scikit-learn's **`BaseEstimator`** and **`TransformerMixin`** classes, we ensured that the code is reusable and compatible with scikit-learn pipelines, allowing efficient and consistent transformations during both training and inference.

Summary

In this chapter, you learned how to effectively manage feature engineering pipelines for both time series forecasting and regression tasks using the powerful tools provided by scikit-learn's Pipeline API. As you've seen, the ability to package preprocessing steps into a unified workflow is crucial for ensuring consistency and reproducibility across both training and inference stages.

For time series forecasting, you explored how to create lagged features, which allow you to capture the temporal dependencies in your data. The custom transformer you built for generating lag features makes it easy to handle shifting time steps, enabling your model to learn from past values. You now understand how to integrate this feature engineering step into a pipeline that can preprocess the data, scale it, and feed it into an XGBoost model, making the entire process seamless and reusable.

When dealing with regression tasks, you faced a different set of challenges, especially in handling missing values in categorical features. By learning how to build a custom transformer to replace missing values with meaningful substitutes—either the string **`Missing`** for high missing rates or the most frequent category for lower rates—you have developed a flexible approach that ensures your model gets clean and usable data. You also applied a variety of preprocessing techniques, from scaling numerical features to one-hot encoding categorical features, creating a comprehensive pipeline that combines both numerical and categorical feature engineering.

Through these examples, you've mastered the ability to build a full-featured pipeline that can not only handle diverse types of data but also ensure that the same transformations are applied during both training and inference. This consistency is critical when deploying models in production because it reduces the risk of introducing errors or inconsistencies between the training data and real-world data the model will eventually face.

Incorporating these pipelines into your workflow will improve the scalability of your machine learning projects. The tools you now have in your toolkit allow you to easily maintain and update models, saving time and ensuring high-quality, reproducible results. Furthermore, you've gained a deeper understanding of how to manage feature engineering in a structured and systematic way, which is essential when handling complex data environments.

By embracing the principles outlined in this chapter, you are now equipped to handle both time series forecasting and regression tasks more efficiently, enabling you to build models that generalize well and perform reliably in production. As you continue to explore and implement machine learning models, these techniques will serve as the foundation for handling the feature engineering process with confidence and precision.

In the next chapter, you will learn about how to deploy your XGBoost model into production. You will also learn about using multithreading and distributed computing to handle large datasets with XGBoost.

13
Deploying Your XGBoost Model

In this chapter, you'll explore how to deploy your XGBoost model into production environments and utilize it for real-time inference. You'll learn about key aspects of model maintenance, including monitoring performance with metrics, which you learned about in *Chapter 11*. You'll also see when and how to retrain models, as well as explore the multithreaded and distributed computing options for optimizing training speed. Lastly, you'll dive into cloud-based deployment using containerization technologies such as Docker.

Here's a breakdown of the key topics we'll cover:

- Training a model using multithreaded and distributed computing with XGBoost
- Packaging a model for production deployment
- Deploying a model using containers
- Servicing your model using the REST API

Technical requirements

You can continue using the same virtual environment that you set up earlier in this book since it already contains the necessary packages. The code samples for this chapter can be found in this book's GitHub repository: https://github.com/PacktPublishing/XGBoost-for-Regression-Predictive-Modeling-and-Time-Series-Analysis.

For this chapter, you'll be using the following tools and libraries:

- Windows Subsystem for Linux (WSL)
- Python 3.9
- XGBoost 1.7.3
- NumPy 1.21.5
- pandas 1.4.2
- scikit-learn 1.4.2
- Seaborn
- Anaconda
- VS Code
- Dask
- Docker
- Flask 3.0
- Gunicorn

> ***USING LINUX FOR DEPLOYMENT***
>
> *When deploying your XGBoost model, it's advisable to use **Linux** for your container environment. Linux is the most widely adopted operating system for distributed computing and offers greater flexibility for deployment across multiple cloud providers. If you're developing on a Windows-based machine, you can use **Windows Subsystem for Linux** (**WSL**) to*

create a Linux environment within your Windows setup.

Training a model using multithreaded and distributed computing with XGBoost

Let's start with a critical aspect of deploying machine learning models: **training time**. As you saw in *Chapter 5*, model training can be a time-consuming process, especially when working with large datasets or when frequent retraining is necessary.

XGBoost offers built-in support for **multithreaded computing**, which can significantly speed up the training process by utilizing multiple CPU cores. In this section, you'll explore how to enable **multithreading** for XGBoost training, which works on both Windows and Linux systems. Then, you'll delve into **distributed computing** options using **Dask** (www.dask.org) on Linux so that you can scale model training across clusters or in cloud environments. Let's begin.

Using XGBoost's multithreaded features

XGBoost has built-in support for multithreaded computing, which allows you to speed up model training by utilizing multiple CPU cores. You can control this by setting the **`nthread`** parameter, which determines the number of threads to use. By default, XGBoost will automatically use the maximum number of available threads.

It's important to note that if you're using Dask, any value you set for **`nthread`** within XGBoost will take precedence over Dask's default configuration. The following example demonstrates how the multithreading parameter works. We'll revisit the *California housing dataset* that you worked with in *Chapter 4*:

1. Create a Python file to demonstrate XGBoost's multithreaded functionality. We've started with a header and named the file **`multithreaded.py`**.

2. Import the necessary modules. You can load the California housing dataset from scikit-learn (**`sklearn`**). You'll also be using **`pandas`**, **`numpy`**, a module called **`time`** to track how long code execution takes, and, of course, **`xgboost`**:

```
import pandas as pd
import numpy as np
import time
import xgboost as xgb
from sklearn.metrics import r2_score
from sklearn import datasets
from sklearn.model_selection import train_test_split
```

3. Now, you can load in the California housing dataset and perform the train-test split using scikit-learn, as you did previously:

```
housingX, housingy = datasets.fetch_california_housing(
    return_X_y=True, as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    housingX,housingy, test_size=0.2, random_state=17)
```

4. Previously, you used the scikit-learn interface for XGBoost. In this example, you'll use the XGBoost API for Python. One difference is that XGBoost uses a data structure called a **DMatrix** to manipulate data. So, the first thing you need to do is convert the dataset from **`numpy`** or **`pandas`** form into **`Dmatrix`** form by using the **`DMatrix`** function and passing in the data and the labels. In this case, we'll be using **`dtrain = xgb.DMatrix(X_train, y_train)`** for the training dataset; do the same for the test dataset:

```
dtrain = xgb.DMatrix(X_train, y_train)
dtest = xgb.DMatrix(X_test, y_test)
```

   Now, the data is in a format that XGBoost can manipulate with efficiency. As mentioned in *Chapter 3*, XGBoost does some sorting and performs other operations on the dataset to speed up execution.

5. At this point, you're ready to train a model using the XGBoost API and the multithreading feature. By default, XGBoost uses the maximum number of threads available. To see the difference, train the model with just two threads, and then increase the maximum number of logical processors you have in your computer. You'll need to use the **`time`** module to get the computation time and print it out so that you can compare the results. First, save the start time with the following line of code:

```
train_start = time.time()
```

6. You can set the training parameters for XGBoost by creating a dictionary with the parameters as key-value pairs. You can configure all the parameters listed in the *Hyperparameters* section of *Chapter 5*. Here, set **`eta = 0.3`** (the learning rate), **`booster = gbtree`**, and **`nthread = 2`**:

```
param = {"eta": 0.3, "booster": "gbtree",
    "nthread": 2}
```

7. Now that the training parameters have been set, you can train the model and save the end of the execution time by using the following code:

```
housevalue_xgb = xgb.train(param, dtrain)
train_end = time.time()
```

8. Print the execution time with a formatted **`print`** statement while subtracting **`train_start`** from **`train_end`** and converting it into milliseconds by multiplying by 10^3:

```
print ("Training time with 2 threads is :{
    0:.3f}".format((train_end - train_start) * 10**3),
    "ms")
```

9. Now, repeat the code and increase the number of threads XGBoost uses by changing the value of **`nthread`**. Since our computer has eight logical processors, I've chosen **`8`**:

```
train_start = time.time()
param = {"eta": 0.3, "booster": "gbtree",
    "nthread": 8}
housevalue_xgb = xgb.train(param, dtrain)
train_end = time.time()
print ("Training time with 8 threads is :{
    0:.3f}".format((train_end - train_start) * 10**3),
    "ms")
```

10. To ensure the model is working as expected, you can make a prediction and check the R^2 value. You can also time the prediction. To make a prediction with the Python API, just call the **`predict`** method on your model and pass the test dataset:

```
pred_start = time.time()
ypred = housevalue_xgb.predict(dtest)
pred_end = time.time()
print ("Prediction time is :{0:.3f}".format((
    pred_end - pred_start) * 10**3), "ms")
xgb_r2 = r2_score(y_true=y_test, y_pred= ypred)
print ("XGBoost Rsquared is {0:.2f}".format(xgb_r2))
```

11. Running this script results in the following output. Please note that the execution time on your computer will be different:

```
Training time with 2 threads is :237.088 ms
Training time with 8 threads is :130.723 ms
Prediction time is :2.012 ms XGBoost
Rsquared is 0.76
```

On our computer, going from two to eight threads sped up training by over 44%. This demonstrates the benefit XGBoost provides with multithreading. Recall that by default, it will use the maximum number of threads available. Next, you'll learn about using XGBoost with distributed compute by using Dask on Linux.

Training with distributed compute

So far, you've used XGBoost on a single local computer. Sometimes, though, a dataset is very large, so to speed up execution, you'll want to share the work across a cluster of machines. From this point onwards in this chapter, we'll be using *Linux* since it's the most common operating system for distributed compute and lets you deploy your model with multiple cloud service providers.

As mentioned earlier in this chapter, you can use **Dask** to enable parallel computing. When using Dask, you need to store your data in **Dask DataFrames**. Dask sets up a computation as a task graph and then calls the **`compute`** method to run it on a scheduler. Schedulers can be single or multithreaded and can be on a single machine or distributed in a cluster. Dask works both on your local machine and with Kubernetes or a cloud service such as AWS or Azure. XGBoost has a Dask feature that you'll be using in this section.

To start using Dask, you'll need to implement an example on your local machine. When you want to move to a Kubernetes cluster or cloud provider such as AWS or Azure, you can comment out the local machine cluster statement and add one for the appropriate service. Details about using the various cloud providers, such as AWS and Azure, are available in the Dask documentation at https://cloudprovider.dask.org/en/latest/index.html.

This example from the Dask documentation (https://docs.dask.org/en/latest/deploying.html) shows how you can easily change from running Dask on your local computer to running Dask in a Kubernetes cluster (**`cluster = KubeCluster()`**):

```
# You can swap out LocalCluster for other cluster types
from dask.distributed import LocalCluster
from dask_kubernetes import KubeCluster
# cluster = LocalCluster()
cluster = KubeCluster()  # example, you can swap out for
                         #Kubernetes
client = cluster.get_client()
```

With this out of the way, you're ready to complete a Dask example on your local computer using **`LocalCluster()`**. Unlike our previous examples, you'll run this one not as a Jupyter notebook but as a Python **`.py`** file. In addition, you'll use Linux when running the code. First, you need to install **`dask`** into a virtual environment using **`conda`**:

1. Open a Linux Terminal window and create a **`conda`** environment for this project. Then, activate the environment by running the following commands:

   ```
   conda create --name xgboost_book_project
   conda activate xgboost_book_project
   ```

   Replace **`xgboost_book_project`** with the name of your virtual environment.

2. Then, run the following command to install Dask:

   ```
   conda install dask
   ```

   You may be prompted to install new packages and update others. Type **`y`** to proceed.

   This will result in the following message:

   ```
   Downloading and Extracting Packages
   Preparing transaction: done
   Verifying transaction: done
   Executing transaction: done
   ```

3. Now, you can create a Python script to test Dask on your local Linux machine. Start a file called **dasktest.py** and import the **pandas**, **numpy**, **time**, and **xgboost** packages, **metrics**, **datasets**, and **train_test_split** from scikit-learn, and various **dask** components. Here's what the top of the script looks like:

```
import pandas as pd
import numpy as np
import time
import xgboost as xgb
from sklearn.metrics import r2_score
from sklearn import datasets
from sklearn.model_selection import train_test_split
from dask import dataframe as dd
from dask.distributed import Client, LocalCluster
from xgboost import dask as dxgb
```

4. Next, you must define a function that makes the Dask calls for training purposes. Pass information about the distributed system to use and the training dataset to the function:

```
def train(client, dtrain):
```

5. The first thing you must do is start a timer so that you can keep track of how long training takes and see the impact of Dask settings such as the number of workers:

```
    train_start = time.time()
```

6. Next, perform training using XGBoost. In the case of distributed computing, XGBoost uses the **approx tree_method**. You can set it directly or use the default, **auto**, which will switch to **approx** when you execute the function:

```
    housevalue_dask = xgb.dask.train(
        client,
        {"verbosity": 2, "tree_method": "approx",
         "objective": "reg:squarederror"},
        dtrain,
        num_boost_round=4,
        evals=[(dtrain, "train")],
    )
```

7. Finish this function by closing out the timer, printing the time, and returning the trained model so that it can be reused later:

```
    train_end = time.time()
    print("Training time with dask is :{
        0:.3f}".format((
            train_end - train_start) * 10**3), "ms")
    return housevalue_dask
```

8. The next function to build is the prediction. You'll want to pass the distributed computing client, the model you trained, and the testing dataset:

```
def predict(client, model, dtest):
```

9. Next, start a timer for the prediction:

```
    pred_start = time.time()
```

10. Then, call **xgb.dask.predict** to make the prediction:

```
    ypred = xgb.dask.predict(client, model, dtest)
```

11. Finish this function by calling **time** to measure how long it takes and using the Dask functionality in XGBoost to predict a value and return the predicted **y** values:

```
    pred_end = time.time()
    print("Prediction time with dask is :{
        0:.3f}".format((pred_end - pred_start) *
        10**3), "ms")
    return ypred
{0:.2f}".format(xgb_dask_r2))
```

12. In this example, you can set the number of workers to 4 with **`n_workers = 4`** and the number of threads per worker (**`threads_per_worker`**) to **`1`**. Feel free to experiment with changing these settings and see the effects on the training and prediction times:

```
if __name__ =="__main__":
    with LocalCluster(n_workers = 4,
        threads_per_worker = 1) as cluster:
            with Client(cluster) as client:
```

13. Now, you'll use Dask to predict California housing values. To do so, you'll be using the California housing dataset that you used previously. First, start the function and load in the California housing dataset from scikit-learn:

```
    # load the California Housing data set from
    #scikit-learn
    housingX, housingy =
    datasets.fetch_california_housing (
        return_X_y=True, as_frame=True)
```

14. Then, perform **`train_test_split`**, as you did previously:

```
X_train, X_test, y_train, y_test = train_test_split(
    housingX,housingy, test_size=0.2, random_state=17)
```

15. Dask requires values to be in a Dask DataFrame. So, you must use the **`from_pandas`** method to convert the output from **`train_test_split`** into Dask DataFrames:

```
            dm_X_train = dd.from_pandas(X_train,
                npartitions=1)
            dm_X_test = dd.from_pandas(X_test,
                npartitions=1)
            dm_y_train = dd.from_pandas(y_train,
                npartitions=1)
            dm_y_test = dd.from_pandas(y_test,
                npartitions=1)
```

16. Then, combine the data with the labels into a **`DaskDMatrix`** structure for both the train and test data:

```
            dtrain= xgb.dask.DaskDMatrix(client,
                dm_X_train, dm_y_train)
            dtest= xgb.dask.DaskDMatrix(client,
                dm_X_test, dm_y_test)
```

17. Now, you're ready to call the **`train`** and **`predict`** functions that you defined previously:

```
            housevalue_dask = train(client, dtrain)
            y_pred = predict(client, housevalue_dask,
                dtest)
```

18. Finish the script by calculating the R^2 value for the Dask XGBoost model:

```
            xgb_dask_r2 = r2_score(y_true=dm_y_test,
                y_pred= y_pred)
```

```
print("XGBoost with dask Rsquared is {
    0:.2f}".format(xgb_dask_r2))
```

19. With that, the script is complete. You can run it by typing the following Linux command in a Terminal:

```
python3 -m dasktest
```

This results in the following output. Recall that the times may vary, depending on how powerful (or not) your computer is:

```
[15:00:50] task [xgboost.dask-0]:tcp://127.0.0.1:33675 got new rank 0
[15:00:50] INFO: /croot/xgboost-split_1675457761144/work/src/data/simple_dmatrix.cc:102:
Generating new Gradient Index.
[0]     train-rmse:1.44755
[1]     train-rmse:1.10970
[2]     train-rmse:0.88376
[3]     train-rmse:0.74196
Training time with dask is :292.709 ms
Prediction time with dask is :63.967 ms
XGBoost with dask Rsquared is 0.55
```

In this section, you used Dask to train and predict an XGBoost model over a cluster of machines, or multiple workers on a single machine. This capability is helpful when a dataset is very large and you wish to speed up execution. Since Dask is flexible, it can run on a local machine, or you can scale your model with Kubernetes or a cloud service such as AWS or Azure. In the next section, you'll learn how to prepare your model so that it can be deployed in a production environment, rather than it having to be run on your local machine.

Packaging a model for production deployment

So far in this chapter, you've been interacting with the models you've developed directly within Python by making calls to functions and methods that train a model or use a model to make a prediction. In a production environment, you won't want to make Python calls directly. You'll either want to create a web application that interfaces with your model, or you'll want to use an **application programming interface** (**API**). An API allows a user to access the functions and methods they need to train and use your model without having to know how to code in Python. Building an API allows a user to call your model from their web application. To better understand this, let's look at the parts of a model that's running in a production environment:

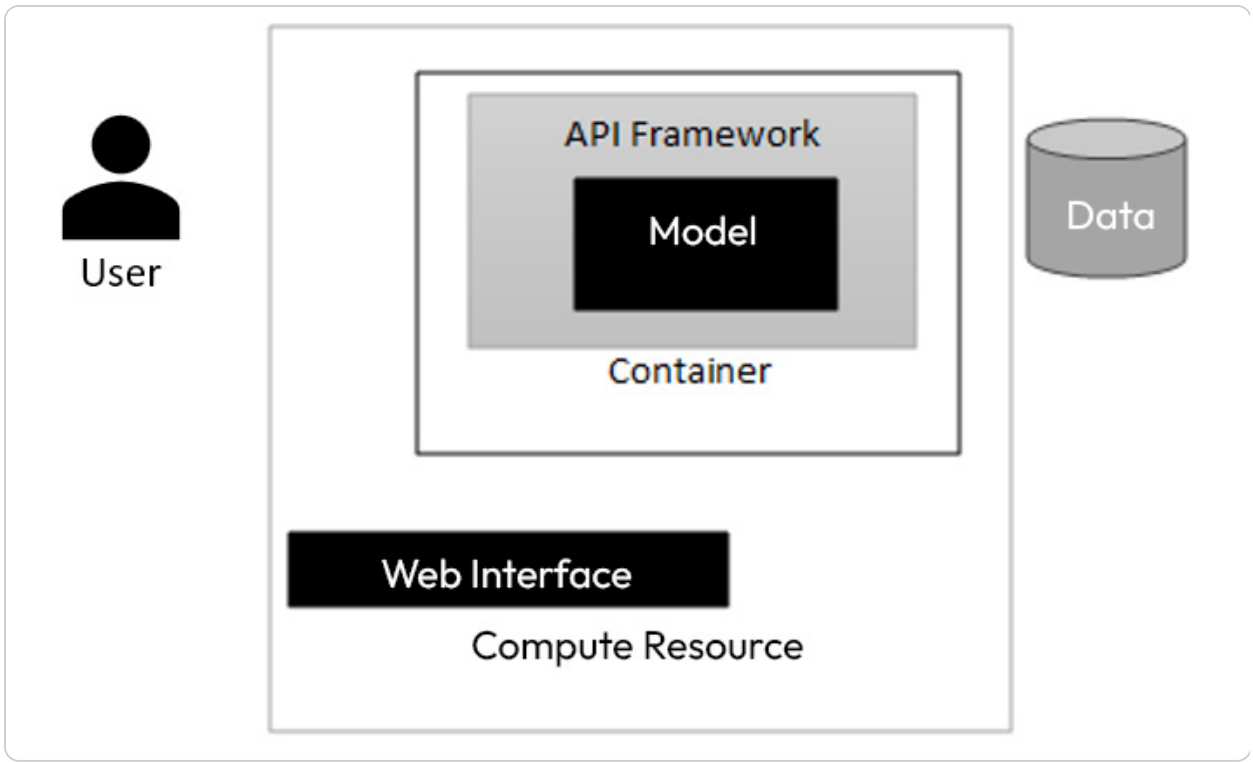


Figure 13.1 – Components of a model running in a production environment

Figure 13.1 shows your model inside an API framework. You'll learn more about this shortly. Both the model and the API framework are inside a container. This container is running on a compute resource that has a web interface. There may also be data storage for your model in a database that can be separate from the compute resource and your model. To access your model, the user makes API calls via the web through the web interface to the API framework running in the container.

But first, let's follow a simpler path to create a web application from a model using Streamlit. Then, later in this chapter, you'll learn how to use the Flask framework to create a **REST API** for your model. REST stands for **Representational State Transfer** and is a standard for how an API should work. After, you'll learn how to use Docker to put your framework and model into a container.

Turning your model into an application with Streamlit

Once you've developed and trained your XGBoost model pipeline, as discussed in *Chapter 12*, the next step is to turn it into an application that allows you to interact with it. **Streamlit**, a Python-based framework, makes this process seamless by providing a simple way to build web applications for machine learning models. With Streamlit, you can deploy your XGBoost model and allow users to input data and make real-time predictions.

Here's a step-by-step approach to turning your model into a web application using Streamlit:

1. Before proceeding, ensure that you have Streamlit installed in your environment. You can install it via **`pip`**:

   ```
   pip install streamlit
   ```

2. Streamlit makes it easy to build an interface where users can input data and view predictions from your model. In the application, you can use **`st.number_input`** for numeric features and **`st.selectbox`** for categorical features. This helps users provide input that matches the model's training data. Here's how you can implement this:

   ```
   import streamlit as st
   ```

```
import pandas as pd
# User Input for Predictions
st.subheader("Make a New Prediction")
input_data = {}
# Loop through numeric features and use number_input for numeric columns
for col in numeric_features:
    input_data[col] = st.number_input(f"Enter {col}:",
        value=float(X[col].mean()))
# Loop through categorical features and use selectbox #for categorical columns
for col in categorical_features:
    unique_values = X[col].unique()
    input_data[col] = st.selectbox(f"Select {col}:",
        options=unique_values)
    input_df = pd.DataFrame([input_data])
    if st.button("Predict"):
        prediction = pipeline.predict(input_df)
        st.write(f"Predicted House Price:
            ${prediction[0]:,.2f}")
```

The preceding code is based on the code that was discussed in *Chapters 7*, *8*, and *12*. For features such as **`LotArea`** and **`YearBuilt`**, the user can input numeric values using the **`st.number_input`** widget. For categorical features such as **`Neighborhood`** and **`HouseStyle`**, users can choose from a predefined list of categories using **`st.selectbox`**. Once the input has been provided, users can click the **Predict** button, which causes the model to make a prediction.

Your pre-trained XGBoost model has been integrated into this application using the pipeline you built earlier in *Chapter 12*, and is run in **`prediction = pipeline.predict(input_df)`**. The prediction is based on the user's input data, and the result is displayed on the application.

3. Now, you're ready to launch the application. To run the application on your local machine, save the Streamlit code in a Python file (for example, **`streamlit_app.py`**) and run the following command in your Terminal:

```
streamlit run streamlit_app.py
```

Once you've executed this command, you'll see the following web page that you can interact with on **`localhost:8501`**:

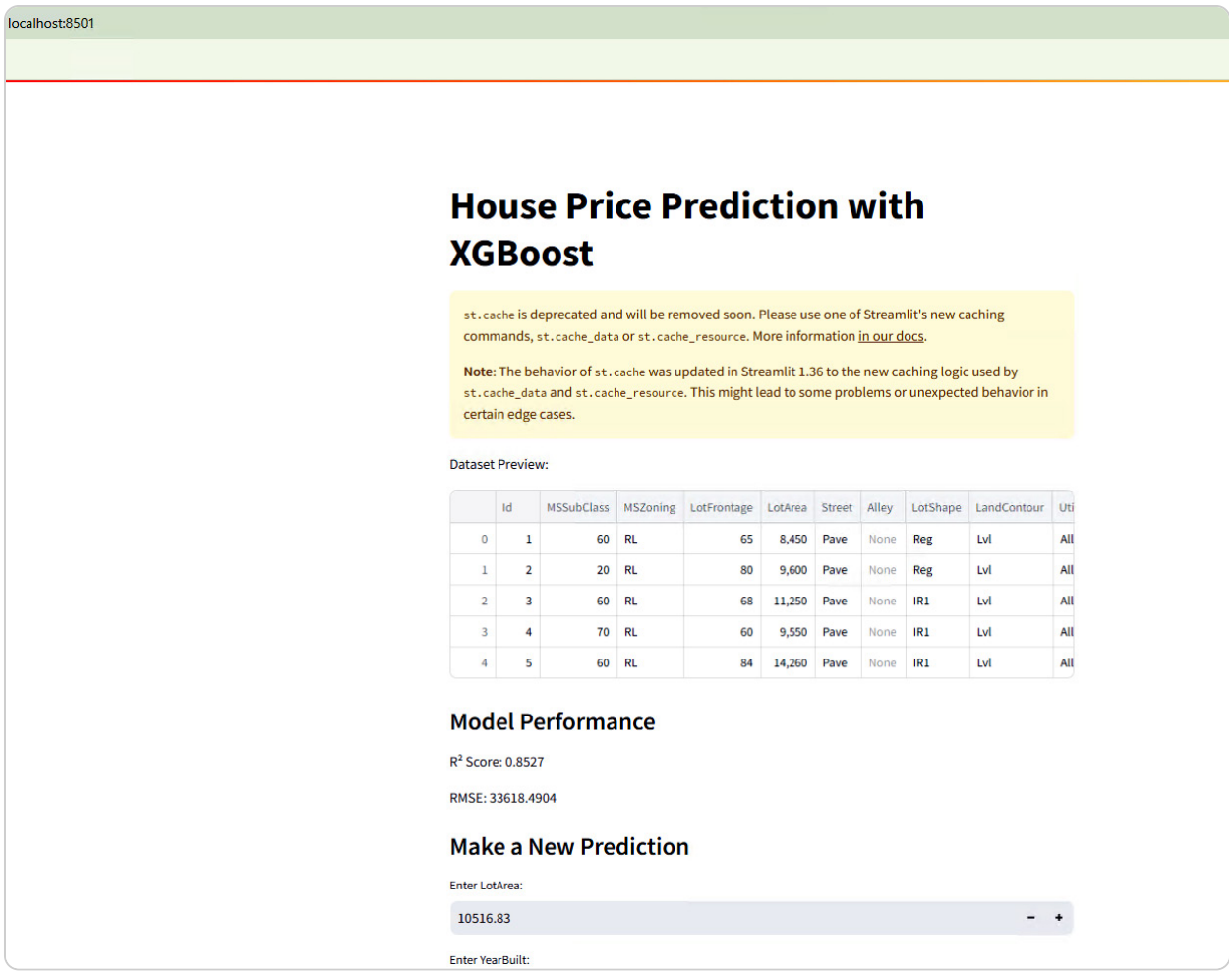


Figure 13.2 – Streamlit web application for the housing prices model

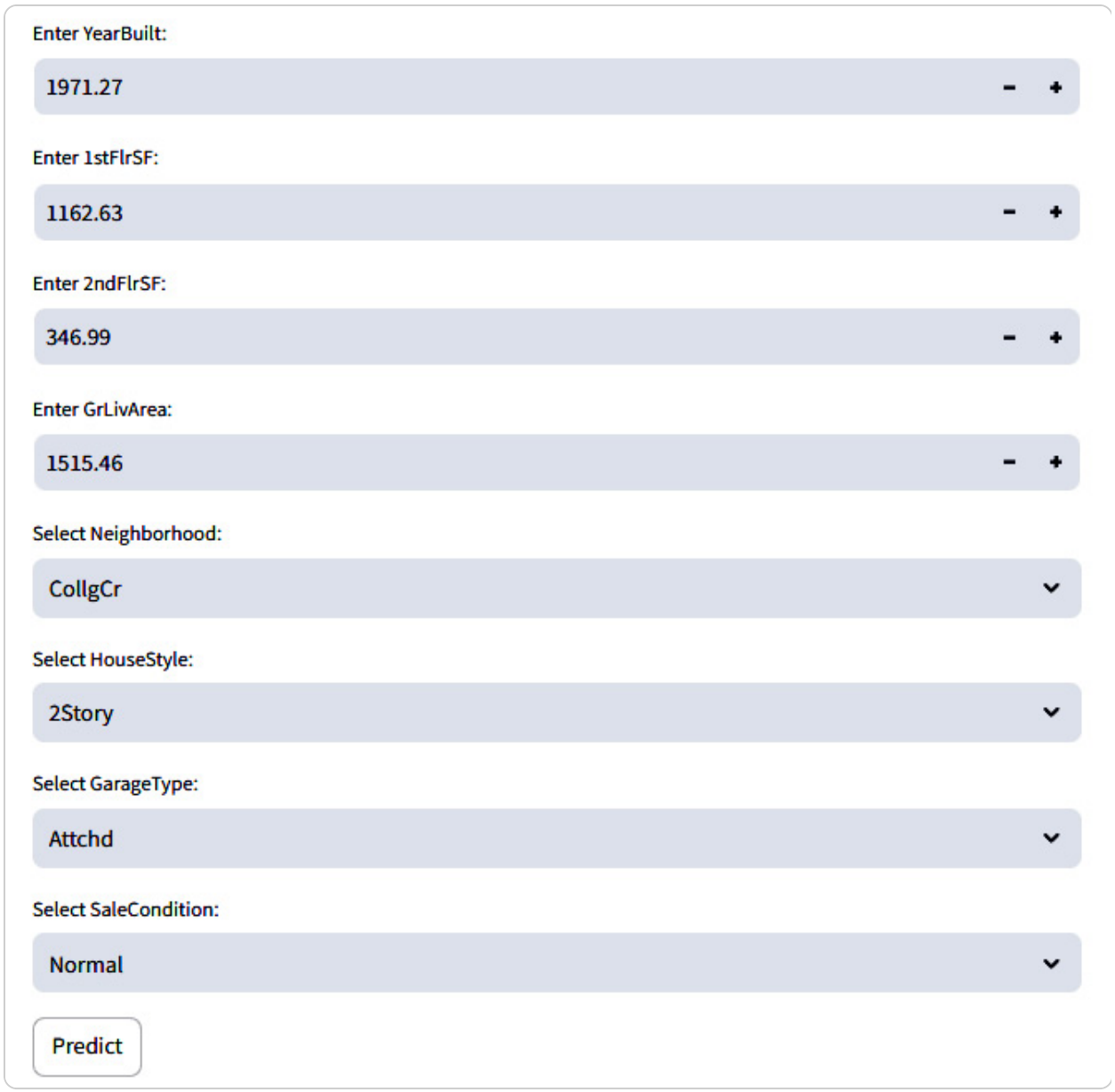


Figure 13.3 – Streamlit application data entry fields

Deploying the application

Once your Streamlit application is ready, it can be deployed on a local machine for internal testing or hosted on cloud platforms for broader access. Services such as Streamlit Cloud, Heroku, and AWS allow you to host your Streamlit application for external users, enabling a fully interactive, web-based machine learning application.

Now that you've built a web application with Streamlit for your model, you'll learn about using the Flask framework to create a **REST API** for your model. You'll use this same Flask framework inside a Docker container.

> ***FASTAPI AS AN ALTERNATIVE TO FLASK AND GREEN UNICORN (GUNICORN)***

> ***FastAPI*** *is a modern web framework for building APIs with asynchronous support natively. While this chapter focuses on deployment with* ***Flask*** *and* ***Gunicorn****, which together enable scalable and performant applications, FastAPI offers an alternative with native asynchronous capabilities, making it ideal for handling high-concurrency applications.*
> *If you're interested in asynchronous frameworks, FastAPI could be a good choice due to its speed and ease of use with async calls. However, for synchronous model inference tasks, Flask and Gunicorn provide reliable scalability and work seamlessly with traditional Python libraries.*
> *FastAPI won't be covered in detail here, but if you'd like to explore it further, here's a recommended resource: FastAPI: Modern, Fast (High-performance) web framework for Python (https://fastapi.tiangolo.com/).*

Creating APIs to call your model

As you develop your model, think about the tasks you'll want to perform with it once it's deployed in production. For each of these tasks, you'll need to build a method or function that can be called via the API.

For your model to work in the cloud, you'll need to save both it and your training pipeline so that they can be moved from your computer to the cloud for processing. You can save these using a **pickle** file. This allows you to easily transport your model from one place to another. To put your model into a pickle file, follow these steps:

1. Begin by installing the **`pickle`** package in your **`conda`** environment:

```
conda install pickle
```

2. In the file where you created the model and the pipeline, import **`pickle`**:

```
import pickle
```

3. Save your pipeline as a pickle file with **`pickle.dump`**:

```
with open('XBGpipeline.pkl', 'wb') as file:
    pickle.dump(pipeline, file)
```

You'll likely need to retrain your model from time to time, either based on changes to the environment in which the model is running or due to degradation in the model's performance. Say, for example, you're creating a classification model, such as the one outlined in our tomato example in *Chapter 11*. When a new classification – perhaps "Supreme" – is introduced into the environment, you need to retrain the model; otherwise, it won't know to place tomatoes into that classification. So, you need to build a method to retrain the model.

Similarly, if any of the fit metrics for your model have degraded, you should retrain it on more recent data. So, another method you could create would check the model fit.

The primary use of the model will also need a method, which is to either classify or predict values based on input data.

This adds up to at least three methods you'll want to create for the following purposes:

1. Retraining your model.
2. Making a prediction.
3. Monitoring model fit metrics.

In this section, you'll use the time series model that you made in *Chapter 9* and build these three functions for it in your Flask environment. Let's start by building a method to retrain the model.

Retraining your model

To retrain the model, you'll need to add any new data you want to use for training to your training dataset. You may have this dataset in storage as a DMatrix, a Flask dataset, or even in a database, depending on your needs. Then, you'll need to call the training function. Finally, you'll save the updated version of the model.

Let's write a method that will retrain the model from *Chapter 9*. This method retrieves the model from a pickle file, refits it on the new data, and saves it back as a pickle file:

```
def retrain_model(model,X_train, y_train):
    #load the model from pickle file, retrain the model and
    #save to pickle
    with open(model, 'rb') as file:
        loaded_model_pickle = pickle.load(file)
    loaded_model_pickle.fit(X_train, y_train)
    base_name, extension = os.path.splitext(file_path)
    new_file_path = f"{base_name}_retrained{extension}"
    with open(new_file_path, 'wb') as file:
        pickle.dump(loaded_model_pickle, file)
```

Now, you have a method that can be called via an API to retrain the time series model. Next, let's make a method that will use the model to make a prediction.

Making a prediction

The second method to create will use the model and either make a prediction or classify data depending on the type of model you've created.

For our time series example, you'll be making a prediction by retrieving the model from the pickle file and then running the **`.predict`** method. The following code loads the model for each prediction. Depending on your needs, you may wish to load the model once – say, when the user starts an API session – and allow multiple predictions to be made:

```
def predict(model, X_test):
    #use the loaded model to predict a result
    with open(model, 'rb') as file:
        loaded_model_pickle = pickle.load(file)
    predictions_pickle = loaded_model_pickle.predict(
        X_test)
    return predictions_pickle
```

In the next section, you'll build a method to monitor how the model is performing.

Monitoring model fit metrics

To monitor model fit, you'll need to create a function that checks the predicted values against a test dataset, just as you did in *Chapter 11*. The metrics you'll use will depend on the problem your model has been developed to address. For example, if you're doing tomato grading, as you did in *Chapter 11*, you could use a confusion matrix and classification report as your metrics. If you're predicting a value, you might select MSE and/or R^2 as your metrics. In this section, you'll use both to evaluate the time series model:

1. Start by making a class to hold the results of model evaluation – that is, **`mse`** and **`r_squared`**:

```
class EvalMetrics:
    def __init__(self):
        self.mse = 0
        self.r_squared = 0
```

2. Next, write a function to evaluate the model. The following code runs the prediction function and then calculates the metrics:

```
def evalmodel(model, X_test, y_test):
    y_pred = predict(model, X_test)
    EvalMetrics.mse = mean_squared_error(y_test,
        y_pred)
    EvalMetrics.r_squared = r2_score(y_test, y_pred)
    return EvalMetrics
```

Here, we've only included the relevant parts of the code. For the complete code, please refer to this book's GitHub repository: https://github.com/PacktPublishing/XGBoost-for-Regression-Predictive-Modeling-and-Time-Series-Analysis. In this section, you created a method to monitor the model's performance. This wraps up the minimum methods needed for you to use an API with your model. Next, you'll learn how to use Flask to call these functions with an API.

Creating an API for your model using Flask

To provide API access to your model, you'll need a framework. **Flask** is a lightweight web framework that allows you to easily create an API for your model. With Flask, you can configure endpoints to make predictions, add new data, and retrain your model on fresh datasets. By utilizing the REST API standard, Flask enables seamless communication between your model and external systems through HTTP/HTTPS requests, allowing for real-time inference and model updates via a web interface.

> ### *FLASK VERSUS DJANGO*
>
> *While **Django** is another popular framework that offers similar functionality, we're using Flask here due to its lightweight nature and flexibility. Unlike Django, Flask is a **microframework**, meaning it doesn't include unnecessary components by default, giving you the freedom to add only the features you need as your project evolves. This makes Flask ideal for smaller projects or scenarios where you're still determining the full scope of the application.*
> *On the other hand, Django is more suited for large-scale production applications that require comprehensive administrative controls, security features, and built-in tools. It offers a more complete set of features out of the box, making it a better choice for complex applications with defined requirements from the outset.*
> *In summary, Flask is perfect for smaller models, rapid prototyping, and projects where you need more flexibility and simplicity. Django is ideal for full-fledged production systems with extensive functionality and robust security requirements. Depending on your project's needs, you can choose the framework that best suits your goals.*

To allow you to send and receive commands to your code, Flask will start a web service. To see how this works, you can build a quick web application to display "Hello World!" using Flask's *Quickstart* guide at https://flask.palletsprojects.com/en/3.0.x/quickstart/. To understand this process, follow these steps:

1. Install Flask using **`conda install`**:

   ```
   conda install flask
   ```

   This results in the following output:

   ```
   Retrieving notices: ...working... done
   Collecting package metadata (current_repodata.json): done
   Solving environment: done
   ## Package Plan ##
     environment location: C:\Users\joyce\anaconda3\envs\xgboost_book_project
     added / updated specs:
       - flask
   Downloading and Extracting Packages
   Preparing transaction: done
   Verifying transaction: done
   Executing transaction: done
   ```

2. Create a new file for the Flask application called **`FlaskTest.py`**:

   ```
   vi FlastTest.py
   ```

3. Inside **`FlaskTest.py`**, import the **`Flask`** class. Using **`Flask`** with a capital letter indicates a class:

   ```
   from flask import Flask
   ```

4. Next, create an application by making an instance of the **`Flask`** class and passing it the **`__name__`** magic method. Flask uses **`__name__`** to know where the Python file that's calling it is located, which, in turn, tells Flask where to find the resources and files it needs to run the application:

```
app=Flask(__name__)
```

5. Tell Flask which URL to respond to with **`route()`**. You can have it respond to all traffic to a particular IP address with **`"/"`** or you can direct it to things such as **`"/docs"`** and **`"/admin"`** as you choose. For this demonstration, set it to **`"/"`**:

```
@app.route("/")
```

6. Finally, tell Flask what you want it to respond with. The default is HTML, so you can format your response string with HTML tags:

```
def hello_world():
    return "<p> Hello World! </p>"
```

7. Run your Flask demonstration application using the following command:

```
python - m flask --app FlaskTest run
```

This produces the following output:

```
* Serving Flask app 'FlaskTest'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a
production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

8. In a browser window, go to **`http://127.0.0.1:5000`**. You'll see the following output in your browser:



Figure 13.4 – Web response from the "Hello World!" Flask application

In your Python window, you'll see messages from Flask responding to the request:

```
127.0.0.1 - - [27/Dec/2023 09:44:45] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [27/Dec/2023 09:44:45] "GET /favicon.ico HTTP/1.1" 404 -
```

Flask will warn you that this is a development server and not for deployment in production. We'll address this warning later in this chapter by helping you put your Flask application into a Docker container with a **Web Server Gateway Interface** (**WSGI**) server.

> ### *USE PRODUCTION SYSTEMS FOR DEPLOYMENT*
>
> *When deploying your model into production, it's important to understand that Flask's development server is not intended for this purpose. The development server is designed for local use and lacks the critical security features and performance capabilities required for a production environment. It's not equipped to handle multiple simultaneous asynchronous requests, which your model will likely need to process in production.*

To ensure your model runs efficiently, securely, and in a scalable and performant way, you should deploy it using Gunicorn and Nginx. Detailed examples of the use of Gunicorn and Nginx are beyond the scope of this book. That said, here's a brief overview of these two networking applications:

- **Gunicorn** is a **WSGIHTTP server** designed for running Python web applications in production. It can handle **multiple concurrent requests** by managing multiple worker processes, making it highly scalable and capable of distributing the load efficiently across your CPU cores.

- **Nginx** is a powerful **reverse proxy server** that sits in front of Gunicorn, handling client requests. It helps manage heavy traffic by serving static files and balancing requests efficiently across the Gunicorn workers. Nginx also adds an important layer of security and performance optimizations, such as handling SSL/TLS encryption, which Flask's development server doesn't support.

Now that you have an idea of how Flask works, you can use it to deploy the methods for each of the commands for your model. We'll do that now:

1. First, modify the file you created with the methods in them. Make sure you have the necessary packages, which include **`Flask`**, **`jsonify`** to send data back to the user, and **`request`** to get data from the URL. Start up Flask with **`app=Flask(__name__)`**:

```
import pandas as pd
from sklearn.metrics import mean_squared_error,
    r2_score
import pickle
import os
from flask import Flask, jsonify, request
app=Flask(__name__)
```

2. If the user wants to retrain the model, they'll call the API with **`/retrain`** and include JSON specifying the new dataset to use. This part of the API loads the model, accepts the JSON file from the URL, and calls the **`.fit`** method. It will output a message stating **`"Model retrained successfully"`** so that the user knows something happened:

```
@app.route("/retrain",methods = ["POST"])
def retrain_model(model="XBGpipeline.pkl"):
    #load the model from pickle file, retrain the model and save to pickle
    with open(model, 'rb') as file:
        loaded_model_pickle = pickle.load(file)
    data = request.get_json()
    df = pd.DataFrame(data)
    # Split the data into features and target variable
    X_train = df.drop('target', axis=1)
    y_train = df['target']
    loaded_model_pickle.fit(X_train, y_train)
    base_name, extension = os.path.splitext(model)
    new_file_path = f"{base_name}_retrained{
        extension}"
    with open(new_file_path, 'wb') as file:
        pickle.dump(loaded_model_pickle, file)
    return jsonify({'message':
        'Model retrained successfully'})
```

3. If the user wants to make a prediction, they can do a similar thing by calling the API with **`/predict`** and including a JSON with the input values for the model to use in the prediction. The following code accepts the JSON and calls the **`.fit`** method, then outputs the prediction as a JSON using the Flask **`jsonify`** method:

```
@app.route("/predict",methods = ["POST"])
def predict(model="XBGpipeline.pkl"):
    if request.method == "POST":
        X_test = request.get_json()
        #use the loaded model to predict a result
        with open(model, 'rb') as file:
            loaded_model_pickle = pickle.load(file)
        try:
            predictions_pickle =
                loaded_model_pickle.predict(X_test)
        except Exception as e:
            predictions_pickle = None
            print("An error occurred:", e)
        return jsonify({'predictions':
            predictions_pickle.tolist()})
```

4. To evaluate the model, the user can call **`/eval`** and pass a JSON with the prediction and actuals in pairs. This code accepts that JSON file and outputs evaluation metrics as a JSON file using a class called **`EvalMetrics`** and **`jsonify`**:

```
@app.route("/eval",methods = ["POST"])
def evalmodel(model="XBGpipeline.pkl"):
    #requires json file with y_pred, y_actual
    data = request.get_json()
    df = pd.DataFrame(data)
    y_pred = df.drop('target', axis=1)
    y_actual = df['target']
    # Create a class to hold the results of model
    #evaluation mse, r_squared
    class EvalMetrics:
        def __init__(self):
            self.mse = 0
            self.r_squared = 0
    EvalMetrics.mse = mean_squared_error(y_pred,
        y_actual)
    EvalMetrics.r_squared = r2_score(y_pred, y_actual)
    return jsonify({'Metrics': EvalMetrics.tolist()})
```

With that, you've used Flask to create an API for your model. With this API, a user can make a prediction, retrain the model, and evaluate the model's performance. Next, you'll learn how to use Docker to containerize your model and API, making it portable and easy to deploy on a cloud service.

Using Docker to put your model into a container

In this section, you'll learn how to containerize a model and API using Docker. Docker is a solution that separates an application from its infrastructure. With Docker, you can package your application in a portable container, which enables you to move it from one computer to another without the need for a lot of configuration work. It also separates your application from anything else running on the same hardware.

A quick overview of Docker

Docker is a client-server application. On the client system, you communicate with the Docker daemon to create and manage images and containers. The daemon runs on the server, which for development is typically the same system. With Docker, you can communicate with registries of images and use them as starting points for your applications. Docker also gives you tools to move your container from system to system. This is illustrated in *Figure 13.5*:

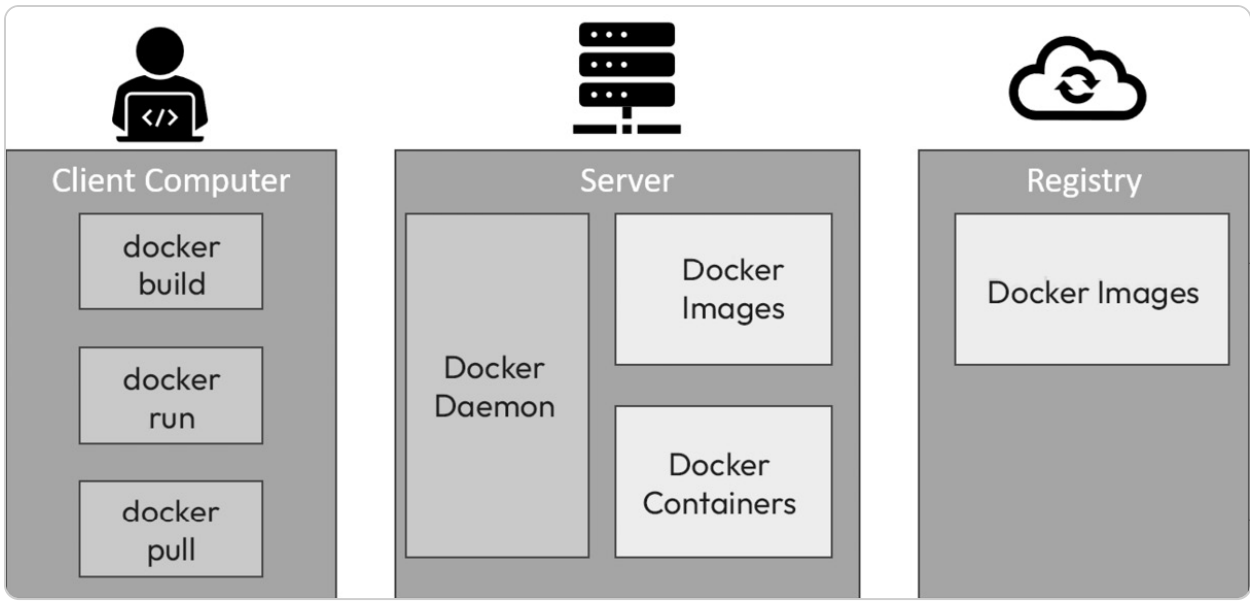


Figure 13.5 – An overview of Docker

Figure 13.5 shows that the various parts of the Docker system are in different locations. On the client computer, Docker gives commands to the Docker daemon running on the server. The **`docker build`** command creates a new image on the server, **`docker run`** executes an image in a container on the server, and **`docker pull`** gets images from a registry. Next, you'll create a Docker container that will hold your model, making your model portable to multiple cloud systems.

Building a container for your model

In this section, you'll learn how to set up the deployment software stack for your model. So far, you've been using Python and XGBoost for training and making predictions, with the model wrapped in a Flask API to allow external access. Now, to move this setup into a production environment, you'll need to add a few more components:

1. First, you'll need a WSGI server to run the Flask API. For this, you'll use Gunicorn, as discussed earlier.
2. Next, you'll need a web proxy server to manage client requests and route them inside the container efficiently. For this purpose, you'll use Nginx.

With these additions, your complete deployment stack will include Python, XGBoost, Flask, Gunicorn, and Nginx, all packaged into a Docker container for smooth production deployment.

Here's what the full stack will look like:

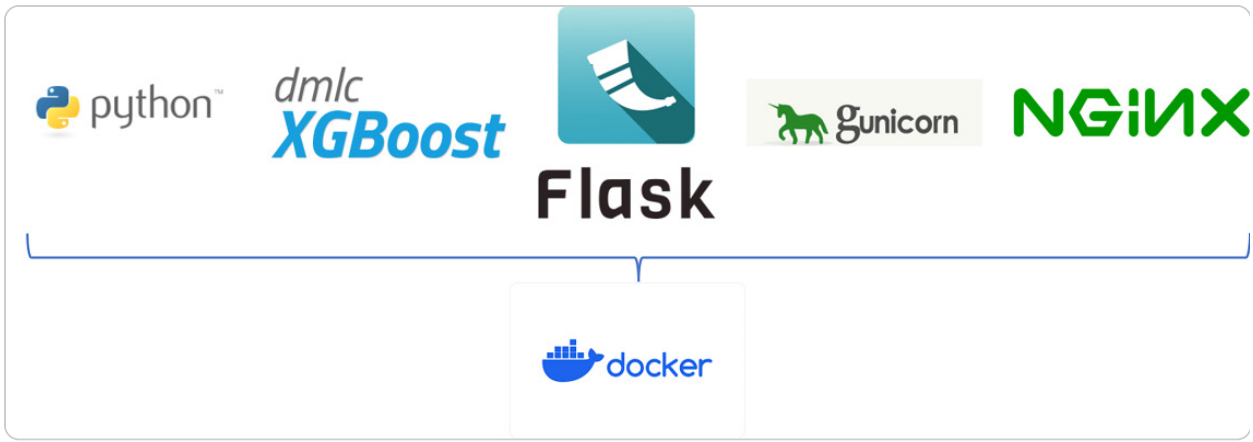


Figure 13.6 – Full software stack for your production model with an API

Figure 13.6, as mentioned previously, shows Python, XGBoost, Flask, Gunicorn, and Nginx all going inside a Docker container. That's exactly what happens, and it's what you'll build in the next section.

Deploying a model using containers

Before you can containerize your application, you need to ensure it runs properly in a production environment. Then, you can build a Dockerfile, a configuration file that reproduces that production environment as a container. You'll complete both the testing and containerization steps on your local system; then, in the next section, you'll learn how to deploy your container to the cloud.

Let's consider an example Dockerfile that includes the pieces you'd need to containerize the Flask API you created earlier in this chapter and the model from *Chapter 9*:

1. First, you'll need to inherit Python from a standardized image. You'll want to match the version to what you've been using – in this case, Python 3.9. The **`-slim`** option reduces the amount of disk space your container takes up:

```
FROM python:3.9-slim
```

2. Next, you'll want to establish a web socket so that it uses **`EXPOSE 8000`**, as well as do some setup so that you can see things happen as your container is run and keep things clean in the container. Here, **`ENV PYTHONUNBUFFERED 1`** tells Docker not to buffer the output so that you see the output of the application in real time. Finally, **`ENV PYTHONDONTWRITEBYTECODE=1`** keeps your container clean by telling Python not to make **`.pyc`** files:

```
EXPOSE 8000
ENV PYTHONUNBUFFERED 1
ENV PYTHONDONTWRITEBYTECODE=1
```

3. Next, you'll want to make sure that the applications inside the container are up to date by running **`apt-get update`**:

```
RUN apt-get update \
&& apt-get install -y --no-install-recommends python3\
&& apt-get clean
```

4. Now, you can make a directory in the container that will hold the API called **`/rest_api`**, set it as the working directory, and copy your Flask API to that directory:

```
RUN mkdir /rest_api
WORKDIR /rest_api
COPY . /rest_api/
```

5. Then, copy over the **`requirements.txt`** file so that all the Python dependencies for your model and API can be installed in the container:

```
COPY ./requirements.txt /rest_api/requirements.txt
```

6. Install the requirements while making sure **`wheel`** is updated first by using **`pip install`**:

```
RUN python3 -m pip --no-cache-dir install wheel
RUN python3 -m pip --no-cache-dir install -r requirements.txt
```

7. At this point, everything is ready, so you can run the **`wsgi`** file for your application using **`gunicorn`**, set up port **`8000`**, and start three **`workers`**. When you use Docker to run your container, the command in CMD is executed, which will start **`gunicorn`** and run your API Flask application:

```
CMD ["gunicorn", "--bind", ":8000", "--workers", "3" "rest_api.wsgi"]
```

With that, you've created a Dockerfile that you can use to deploy a containerized model to a registry and then retrieve the container on any cloud resource that runs Docker. Next, we'll discuss some aspects you should consider when deploying a container in the cloud.

Considerations when deploying a container in the cloud

The big cloud services (Google, Amazon, and Microsoft Azure) can run your Docker container and often have a free tier that you can use if you don't require storage or shared services. You'll need to copy your container Dockerfile to the cloud service and build the image in the cloud service. There are multiple cloud provider options you can use with a Docker container, each of which has its specific set of steps. For this reason, we won't cover it here. Feel free to try out the big cloud services or other containers as service providers. However, we will cover some important considerations when moving to production.

> ### *CONSIDERING RISK AND WEB SECURITY WHEN DEPLOYING A MODEL*
>
> *Deploying a machine learning model into a production environment is a complex task that should be approached with caution. Beyond ensuring that your model performs as expected, any system that's exposed to the web is vulnerable to risks such as cyberattacks, hacks, and user errors. To minimize these risks, it's critical to follow DevOps best practices, which include performing rigorous input validation on all data received through your web API and thorough testing before deployment.*

Ignoring best practices when deploying a model can expose sensitive data, compromise the security of your system, or lead to operational failures. While web security and operational concerns are beyond the scope of this book, we highly recommend diving into the following resources to gain a solid understanding of DevOps and MLOps for secure, scalable deployments:

- *The Phoenix Project: A Novel about IT, DevOps, and Helping Your Business Win*, by Gene Kim, et al.
- *DevOps for Web Development: Achieve Continuous Integration and Continuous Delivery of Your Web Applications with Ease*, by Mitesh Soni.
- *Engineering MLOps*, by Emmanuel Raj.

By leveraging the insights from these books, you can develop a deeper understanding of how to secure and optimize the deployment of machine learning models while minimizing risk.

In the next section, we'll discuss how you can service your model once it's been deployed using the REST API.

Servicing your model using the REST API

Once your model has been deployed and your container is running, you can use REST API commands to access it. These commands interface with the methods you created earlier in this chapter in the *Creating API calls to your model* section. In this section, you'll use REST API commands to interface with the time series model from *Chapter 9*. Let's begin by checking how the model is performing.

Monitoring model performance

To test how your model is performing, open a browser window and navigate to the URL that's serving the model – for example, say your model is hosted at https://example.com. Note that this is completely made up and this URL won't work. To check model performance, the appropriate REST command to use would be **`GET`** (https://example.com/api/v1/metrics). You can use the Python **`requests`** module to interact with the API programmatically. So, let's do that:

1. Begin by importing the **`requests`** module:

```
import requests
```

2. Next, form the URL to **`GET`** the model performance metrics:

```
url = https://example.com/api/v1/metrics
```

3. Now, set up headers so that you can pass your API key:

```
headers = { "api_key": "YOUR_API_KEY" }
```

4. Then, create a variable to store the response:

```
response = requests.request("GET",url,
    headers=headers)
```

5. Finally, print the response from the API:

```
print(response.text)
```

Next, you'll learn how to check the model metrics using the API.

Determining when to retrain

Knowing when to retrain your XGBoost model is key to keeping its predictions accurate over time. By tracking performance metrics such as accuracy, precision, recall, and RMSE, you can spot signs of decline. Changes in data patterns, feature distributions, or drops in prediction quality often signal that it's time for retraining. Regularly comparing recent data to your original training data and monitoring these metrics helps you set thresholds that indicate when the model is no longer performing well. This ensures your model stays reliable and accurate in production.

Retraining your model

Retraining your model means updating it with new data to reflect changing trends and patterns. This process begins by gathering fresh, labeled data that represents the current environment in which your model will be working. From there, you'll preprocess this new data, merge it with existing datasets, and adjust hyperparameters to maintain or even improve performance. Once the model has been retrained, it's crucial to validate it against your benchmarks before redeploying it to production, ensuring it continues to provide accurate, reliable predictions.

In this section, we discussed how you can use the REST API to interact with your model once it's been deployed into production. You saw how to perform common maintenance tasks such as monitoring performance metrics. Now, let's recap what we learned in this chapter.

Summary

In this chapter, you learned how to deploy your XGBoost model into production. You built an API to allow users to access the model through a web interface and make predictions, known as inference. You also created functions to manage your model by adding new data and retraining it when needed. Additionally, you explored how to monitor model performance using the metrics discussed in *Chapter 11*, and you learned how to determine when retraining is necessary.

You started this chapter by exploring multithreaded training options and wrapped up with cloud-based deployment using containers.

At this point, you've gained a solid understanding of how to use XGBoost for various types of data. You've had hands-on experience with the XGBoost Python API through practical use cases in classification, regression, and time series analysis. You've also practiced testing, evaluating, and deploying your models into production.

We hope this book has provided you with a strong understanding of the XGBoost algorithm, helped you successfully install and use the XGBoost API, and guided you through the process of preparing datasets, training models, making predictions, and evaluating and deploying them using **Python** and **scikit-learn**. We wish you continued success in your journey with machine learning and XGBoost!

Index

As this ebook edition doesn't have fixed pagination, the page numbers below are hyperlinked for reference only, based on the printed edition of this book.

A

accuracy 220

Anaconda 12

APIs, to call model

  creating 262, 263

  model fit metrics, monitoring 264

  prediction, making 263

  retraining 263

application programming interface (API) 258

  creating, for model with Flask 265-270

Area Under the Curve (AUC) 59, 225

Autoregressive Integrated Moving Average (ARIMA) 178

B

bagging 6

  versus boosting 6, 7

base models 6

bias 6

bimodal 87

binary classification model 218, 219

  defect classifier, evaluating on imbalanced data 222-225

  evaluating 225

  graphical analysis of 225-227

  metrics 219-222

boosting 6, 32

  versus bagging 6, 7

C

California housing dataset

SHAP, using with 200-203
categorical encoding
importance 140, 141
categorical variables
encoding, with simple technique 133-138
identifying, with missing values 110, 111
missing values, correcting with threshold selection and imputation 111-114
missing values, handling 108-110
classification and regression models
model performance, evaluating 218
classification and regression tree (CART) model 4, 30, 31, 63
classification decision tree models 4, 5
compressed sparse row (CSR) 143
confusion matrix
graphical analysis of 225-227
container
building, for model 271
used, for deploying model 271, 272
containers, in cloud
deploying, considerations 273
continuous data
4-pseudo sigma filter, for
removing outliers 90-92
cleaning 90
normalizing 95, 96
standardizing 92-94
convolution neural networks (CNNs) 9
Coordinated Universal Time (UTC) 87
count (frequency) encoding 151-153

D

Dask 253
URL 250
Dask DataFrames 253

data
exploring 14
ingesting 14
preparing, for modeling 21
data cleaning 22
duplicate data, handling 8
imbalanced data, handling 9
inconsistent data, handling 9
missing data, handling 8
outliers, handling 9
data-cleaning methods 88, 89
continuous data, cleaning 90
date data 99
ordinal and nominal data, cleaning 96, 97
data exploration 86-88
data leakage 240
datasets
exploring, with graphs 44-49
date-based features
adding 182, 183
date data 99
reformatting 99, 100
decision-tree-based model
need for 80
versus deep learning model 80
DecisionTreeRegressor
parameters 70, 71
deep learning (DL) 60
use cases 80
deep learning model
need for 80
versus decision-tree-based model 80
distributed computing 250
used, for training model 253-258

used, for training model with XGBoost 250

Django

versus Flask 265

Docker

overview 270

using, to put model into container 270

E

ensemble decision tree models

XGBoost, comparing with 38

ensemble model 64

versus single CART model 64

ensemble modeling 6, 7

ensemble tree-based model

need for 78

ensemble tree model

need for 79

versus simple model 79

exogenous variables 178

explainability

significance 196

Explain Like I'm 5 (ELI5)

applying, for model interpretation 205

need for 205

XGBoost model, explaining with 206, 207

exploratory data analysis (EDA)

categorical variables analyzing 105, 106

data analysis, missing 105

dataset and data description 104

numerical variables, analyzing 105

performing 104

review 104

target variable, analyzing 105

variable data types 105

F

F1 score 221

false negatives (FN)s 220

false positives (FP) 220

FastAPI 262

feature engineering 58, 88

  performing, on house prices dataset 106

  techniques 9

feature engineering techniques, for categorical features 128

  categorical variables 128

  order mappings 128

  rare labels, identifying 131-133

feature engineering techniques, for numerical features

  feature scaling 123

  log transformation rationale 118-120

  log transform, performing 117, 118

  skewed variables, handling by transformation into binary variables 120-122

feature engineering techniques, for temporal features

  temporal variables, handling 125

  time differences, deriving 126, 127

feature scaling 123

  min-max scaling, implementing 123-125

Flask 262

  versus Django 265

future predictions

  improving 193

future values

  forecasting 189

  forecast, visualizing 192, 193

  future time, forecasting 190, 191

  input data preparation, for forecasting 191

  iterative forecasting 190

G

gbm 36

Gini impurity 33

gradient boosted trees 30

  boosting 32

  CART model 30, 31

  gradient descent 32, 33

  problems, tackling 33, 34

  usage, considerations 39

gradient boosting

  used, for comparing XGBoost 74-78

gradient boosting model features

  explaining, with PDPs 207-210

grid search

  improving 194

ground truth 27

Gunicorn 262

H

high cardinality 144

house prices dataset

  citation 104

  feature engineering, performing on 106, 107

  separating, into training and testing sets 107, 108

housing dataset

  used, for comparing models 65, 66

  used, for comparing model results 82, 83

hyperparameters 80

I

imbalanced data

  correcting, by sampling dataset 101

  data for class, enhancing by imputing values 102

  handling 101

imbalanced-learn Python library

  reference link 102

integer encoding 148-150

interpretability

  significance 196

Iris dataset

  exploring, with graphs 16-19

  ingesting 14-16

iterative forecasting 186, 190

K

kurtosis 118

L

lag features 241

  creating 179-182

linear regression

  need for 69

  used, for comparing XGBoost 66-69

Linux

  using 250

Local Interpretable Model-agnostic Explanations (LIME)

  implementing, for model interpretation 203

  need for 203

  using, to interpret XGBoost predictions 203-205

  versus SHAP 205

M

machine learning (ML) 101

  data cleaning 8, 9

  data engineering 7, 8

  data preparation 7, 8

  feature engineering 9

  fundamentals 3

macro average 228

mean absolute error (MAE) 187

mean encoding 157-160

mean squared error (MSE) 54, 187, 243

micro average 228

microframework 265

min-max scaling 123

model

  container, building 271

  deploying, with containers 271, 272

  packaging, for production deployment 258

  putting, into container with Docker 270

  turning, into application with Streamlit 259, 260

model interpretation

  LIME, implementing for 203

  PDPs, exploring 207

  SHAP, exploring for 200

model parameters

  comparing, for Iris and housing datasets 59

model performance

  evaluating, of classification and regression models 218

  evaluating, to measure model fit 232, 234

  training and inference time, measuring 234, 235

model, with Flask

  API, creating for 265-270

model, with REST API

  determining, when to retrain 274

  performance, monitoring 274

  retraining 275

  servicing 273

moving averages 183

moving statistics 183

multi-class classifier metrics 228

  macro average 228-231

  micro average 228-231

  weighted average 228-231

multicollinearity 142

multimodal 87

multiple concurrent requests 267

multithreaded computing 250

  used, for training model with XGBoost 250

N

native API

  using 214

native Python APIs

  versus scikit-learn API 215-218

natural language processing (NLP) 97

neural networks (NNs) 60

numerical variables

  binary encoding, for missing data 115-117

  identifying, with missing data 114

  imputation, with mean values 115-117

  missing values, handling 114

O

one-hot encoding 142, 143

  using, with frequent categories 144-147

One-Versus-One (OVO) 228

One-Versus-Rest (OVR) 228

ordered integer encoding 153-157

order mappings 128

  of variables, with ordered values 130, 131

  of variables, with quality assessments 129

ordinal and nominal data

  cleaning 96, 97

  spelling correction 97

overfitting 65

P

partial dependence plots (PDPs)

  exploring, for model interpretation 207

gradient boosting model features, explaining with 207-210
need for 207
pickle file 262
pipeline
regression feature engineering 244
time series feature engineering 241
using, to avoid data leaks 240
using, with regression data 244
using, with time series data 241-243
pipeline, using with regression data
dataset, preparing 245
missing values, handling 244, 245
model, evaluating 247
pipeline, building 246
precision 220
predictive modeling
data, preparing 49-51
principal component analysis (PCA) 9
probability ratio encoding 161-165
production deployment
model, packaging for 258
Python
data pipeline, implementing 241

Q

quick start code
housing dataset, downloading 42-44
switching, to different dataset 42

R

random forest algorithm 34
random forest model
used, for comparing XGBoost 74-78
random search 194
rare categories

engineering 167-173

recall 220

Receiver Operating Characteristics Curve (ROC) 225

regression decision tree models 4, 5

regression feature engineering

in pipeline 244

regression RMSE R^2

metrics 232

regression tree (CART) model

need for 73, 74

used, for comparing XGBoost 70-72

Representational State Transfer (REST API) 258, 261

residuals 55

reverse proxy server 267

ROC-AUC curves

graphical analysis of 225-227

rolling mean 183

rolling standard deviation 184

rolling statistics 183

root mean square error (RMSE) 51, 218, 232

R-squared (R^2) 232, 243

S

scikit-learn 36

scikit-learn API

using 214

versus native Python API 215-218

scikit-learn's Pipelines 240

Seasonal ARIMA (SARIMA) 178

SHapley Additive exPlanations (SHAP)

exploring, for model interpretation 200

need for 200

using, with California housing dataset 200-203

versus LIME 205

SHAP summary plot 202

SHAP waterfall plot 201

simple model

  benefits 79

  need for 79

  versus ensemble tree model 79

single CART model 64

  versus ensemble model 64

skewed variables

  transforming, into binary variables 120-122

skewness 118

sklearn.preprocessing package 89

standardizing 92-94

Streamlit

  used, for turning model into application 259, 260

Streamlit application

  deploying 261, 262

supervised learning 4

Synthetic Minority Oversampling TEchnique (SMOTE) 102

T

tall data 39

time series data

  characteristics and components 176, 177

  cyclical pattern 177

  handling, considerations 185, 186

  pipeline, using 241

  pipeline, using with 241-243

  random variation 177

  seasonality 177

  trend 177

time series data preparation, for XGBoost 179

  date-based features, adding 182, 183

  lag features, creating 179-182

statistics, rolling 183-185

time series forecasting 175

XGBoost, using 178

time series model

evaluating 243, 244

time series prediction 175

training dataset 112

training time 250

train-test split 8

train-test separation 107

transformed data 102

true negatives(TNs) 220

true positives (TP) 220

U

unsupervised learning 4

W

Web Server Gateway Interface (WSGI) 267

weighted average 228

Weight of Evidence (WoE) 141

encoding 165-167

wide data 39

Windows Subsystem for Linux (WSL) 250

WSGIHTTP server 267

X

XGBoost 29, 30, 175

benefits and features, exploring 30

comparing, to gradient boosting 74-78

comparing, to linear regression 66-69

comparing, to random forest model 74-78

comparing, to regression tree (CART) model 70-72

comparing, with ensemble decision tree models 38

comparing, with random forest models 38, 39

distributed computing, used for training model with 250
feature importance scores, implementing 196-200
gradient boosted trees 30
model training 51, 52
multithreaded computing, used for training model with 250
multithreaded features, using 250-253
predictive model settings 51, 52
setting up 22-24
time series data, preparing 179
training 22-24
used, for making prediction 24-59
using, for time series forecasting 178, 179
XGBoost algorithm 34
improvements, to addressing memory limitations and speed computation 36, 37
overfitting, handling 34-36
sparse data, handling 35, 36
XGBoost API
native API, using 214
scikit-learn API, using 214
scikit-learn API, versus native Python API 215-218
working with 214
XGBoost evaluation metrics 24
XGBoost hyperparameters 236, 237
tuning 235, 236
tuning, practical steps 237, 238
tuning, to improve model fit and efficiency 235
XGBoost learning parameters
reference link 80
XGBoost model
evaluating 187, 189
explaining, with ELI5 206, 207
training 186, 187
XGBoost package
downloading 12, 13

installing 12, 13

XGBoost predictions

LIME, using to interpret 203-205

XGBoost regression hyperparameters

setting 80-82

x-y plots

used, for finding relationships 19, 21



packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

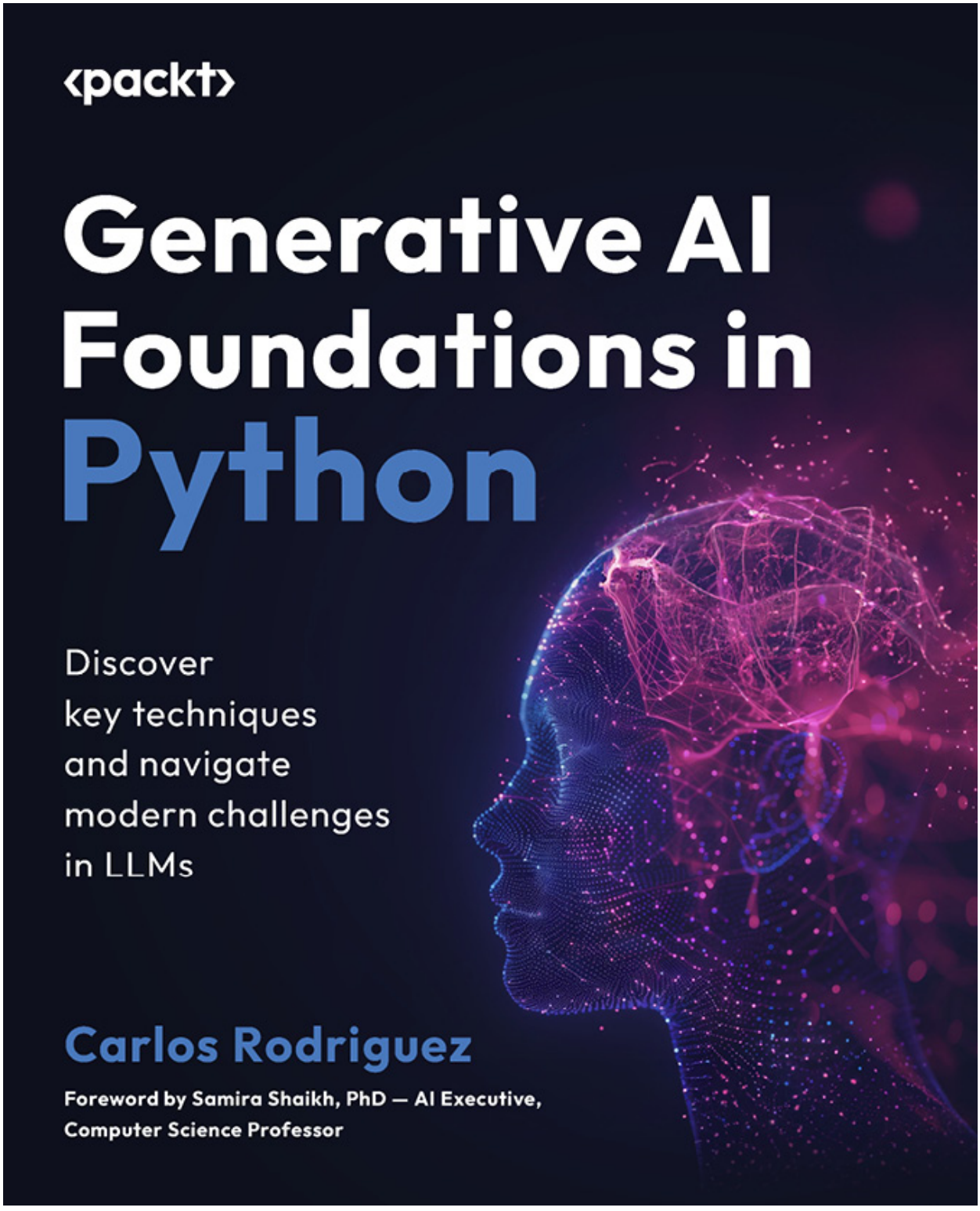
- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packtpub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

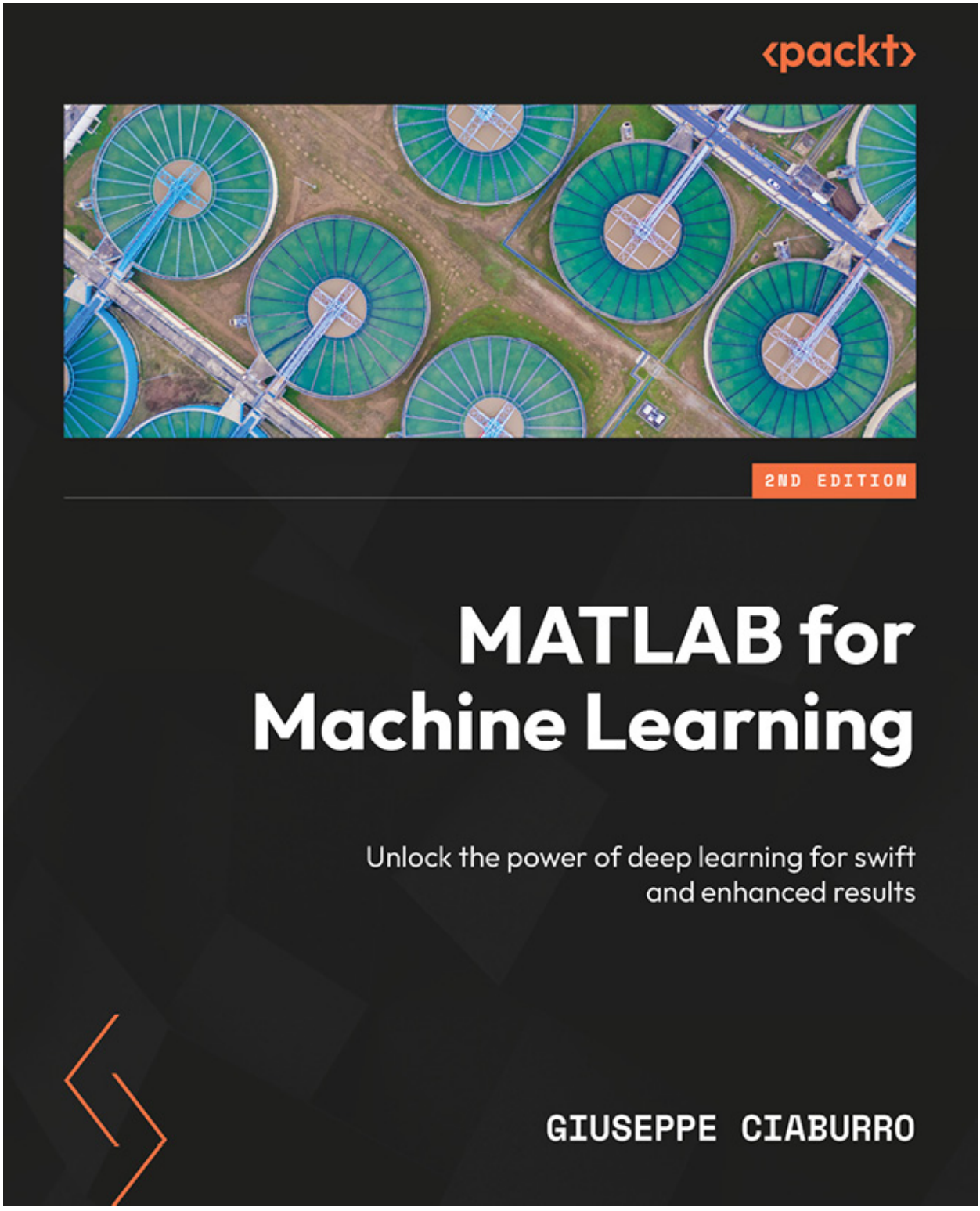


Generative AI Foundations in Python

Carlos Rodriguez

ISBN: 978-1-83546-082-5

- Discover the fundamentals of GenAI and its foundations in NLP
- Dissect foundational generative architectures including GANs, transformers, and diffusion models
- Find out how to fine-tune LLMs for specific NLP tasks
- Understand transfer learning and fine-tuning to facilitate domain adaptation, including fields such as finance
- Explore prompt engineering, including in-context learning, templatization, and rationalization through chain-of-thought and RAG
- Implement responsible practices with generative LLMs to minimize bias, toxicity, and other harmful outputs



MATLAB for Machine Learning - Second Edition

Giuseppe Ciaburro

ISBN: 978-1-83508-769-5

- Discover different ways to transform data into valuable insights
- Explore the different types of regression techniques
- Grasp the basics of classification through Naive Bayes and decision trees
- Use clustering to group data based on similarity measures
- Perform data fitting, pattern recognition, and cluster analysis
- Implement feature selection and extraction for dimensionality reduction
- Harness MATLAB tools for deep learning exploration

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *XGBoost for Regression Predictive Modeling and Time Series Analysis*, we'd love to hear your thoughts! If you purchased the book from Amazon, please click here to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



https://packt.link/free-ebook/9781805123057

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Contents

1. XGBoost for Regression Predictive Modeling and Time Series Analysis
2. Foreword
3. Contributors
4. About the authors
5. About the reviewers
6. Preface
    1. Who this book is for
    2. What this book covers
    3. To get the most out of this book
    4. Download the example code files
    5. Conventions used
    6. Get in touch
    7. Share Your Thoughts
    8. Download a free PDF copy of this book
7. Part 1:Introduction to Machine Learning and XGBoost with Case Studies
8. Chapter 1: An Overview of Machine Learning, Classification, and Regression
    1. Fundamentals of machine learning
    2. Supervised and unsupervised learning
    3. Classification and regression decision tree models
    4. Ensembled models: bagging versus boosting
    5. Data preparation and data engineering
        1. Data cleaning
        2. Feature engineering
    6. Summary
9. Chapter 2: XGBoost Quick Start Guide with an Iris Data Case Study
    1. Technical requirements
    2. Downloading and installing the XGBoost package
    3. Ingesting and exploring data
        1. Ingesting the Iris dataset
        2. Exploring the dataset by making graphs
        3. Looking for relationships with x-y plots
    4. Preparing data for modeling
    5. Setting up and training XGBoost
    6. Using XGBoost to make a prediction
    7. What's next?
    8. Summary
10. Chapter 3: Demystifying the XGBoost Paper
    1. Examining the paper—XGBoost: A Scalable Tree Boosting System—at a high level
    2. Exploring the features and benefits of XGBoost
        1. Gradient-boosted trees
        2. Problems to tackle that would improve the gradient-boosted tree algorithm
    3. Understanding the XGBoost algorithm
        1. Addressing problems – sparse data, overfitting
        2. Improvements to address memory limitations and speed up computation
    4. Comparing XGBoost with other ensemble decision tree models
        1. XGBoost compared with random forest models

        2. When to use gradient-boosted trees
    5. Summary
11. Chapter 4: Adding on to the Quick Start – Switching out the Dataset with a Housing Data Case Study
    1. Technical requirements
    2. Switching the quick start code to a different dataset
        1. Downloading the housing dataset
        2. Exploring the dataset by making graphs
    3. Preparing data for predictive modeling
    4. XGBoost predictive model settings and model training
    5. Making a prediction using XGBoost
    6. Comparing model parameters for the Iris and housing datasets
    7. Summary
12. Part 2: Practical Applications – Data, Features, and Hyperparameters
13. Chapter 5: Classification and Regression Trees, Ensembles, and Deep Learning Models – What's Best for Your Data?
    1. Technical requirements
    2. When to use ensemble models versus single CART models
    3. Comparing models with the housing dataset
    4. Comparing XGBoost to linear regression
        1. When to use linear regression
    5. Comparing XGBoost to CART
        1. When to use CART
    6. Comparing XGBoost to gradient boosting and random forest models
        1. When to use ensemble tree-based models
        2. When to use simple models versus ensemble tree models
    7. When to use deep learning models versus decision-tree-based models
    8. Setting XGBoost regression hyperparameters
    9. Results from comparing models with the housing dataset
    10. Summary
14. Chapter 6: Data Cleaning, Imbalanced Data, and Other Data Problems
    1. Technical requirements
    2. Real-life data is never clean
    3. What to look for when exploring data
    4. Data-cleaning methods
        1. Cleaning continuous data
        2. Cleaning ordinal and nominal data
        3. Correcting spelling in a column
        4. Reformatting date data
    5. Handling imbalanced data
        1. Correcting imbalanced data by sampling the dataset
        2. Enhancing data for a class by imputing values
    6. Summary
15. Chapter 7: Feature Engineering
    1. Technical requirements
    2. A review of exploratory data analysis
        1. Dataset and data description
        2. Performing EDA
    3. Performing feature engineering on the house prices dataset
        1. Separating the dataset into training and testing sets

    2. Handling missing values for categorical variables
    3. Handling missing values for numerical variables
4. Common feature engineering techniques for numerical features
    1. Performing a log transform
    2. Log transformation rationale
    3. Handling skewed variables by transforming them into binary variables
    4. Scaling features
5. Common feature engineering techniques for temporal features
    1. Handling temporal variables
    2. Deriving time differences
6. Common feature engineering techniques for categorical features
    1. Understanding categorical variables
    2. Order mappings – capturing order or related quality
    3. Identifying rare labels for the rest of the categorical/nominal variables
    4. Encoding categorical variables using a simple technique
7. Summary

16. Chapter 8: Encoding Techniques for Categorical Features
    1. Technical requirements
    2. Why is categorical encoding necessary?
    3. One-hot encoding
    4. One-hot encoding with frequent categories
    5. Integer encoding
    6. Count or frequency encoding
    7. Ordered integer encoding
    8. Mean encoding
    9. Probability ratio encoding
    10. Weight of Evidence (WoE) encoding
    11. Engineering rare categories
    12. Summary
17. Chapter 9: Using XGBoost for Time Series Forecasting
    1. Technical requirements
    2. Time series data – characteristics and components
    3. Why use XGBoost for time series forecasting?
    4. Preparing time series data for XGBoost
        1. Creating lag features
        2. Adding date-based features
        3. Rolling statistics
    5. Practical considerations for handling time series data
    6. Training the XGBoost model
    7. Evaluating the model
    8. Forecasting future values
        1. What is iterative forecasting?
        2. Forecasting future time steps
        3. Preparing input data for forecasting
        4. Visualizing the forecast
    9. Improving future predictions
    10. Summary
    11. References
18. Chapter 10: Model Interpretability, Explainability, and Feature Importance with XGBoost
    1. Technical requirements

2. Why interpretability and explainability matter
3. Implementing XGBoost's feature importance
4. Exploring SHAP for model interpretation
    1. Why SHAP is useful
    2. Using SHAP with the California housing dataset
5. Implementing LIME for model interpretation
    1. Why use LIME?
    2. Using LIME to interpret XGBoost predictions
    3. LIME versus SHAP
6. Applying ELI5 for model interpretation
    1. Why use ELI5?
    2. Explaining an XGBoost model with ELI5
7. Exploring PDPs for model interpretation
    1. Why use PDPs?
    2. Explaining gradient boosting model features with PDPs
8. Summary

19. Part 3: Model Evaluation Metrics and Putting Your Model into Production
20. Chapter 11: Metrics for Model Evaluations and Comparisons
    1. Technical requirements
    2. Working with the XGBoost API
        1. Using the native API
        2. Using the scikit-learn API
        3. Comparing the scikit-learn and native Python APIs
    3. Evaluating model performance of classification and regression models
        1. Binary classification
        2. Binary classifier metrics – accuracy, precision, recall, and F1 score
        3. Graphical analysis of binary classification, ROC-AUC curves, and confusion matrix
        4. Multi-class classifier metrics – micro, weighted, and macro averages
        5. Metrics for regression RMSE R2
        6. Evaluating how well a model fits the data summary
        7. Model performance, training, and inference time
    4. Tuning XGBoost hyperparameters to improve model fit and efficiency
        1. What is hyperparameter tuning?
        2. Key XGBoost hyperparameters
        3. Practical steps for tuning
    5. Summary
21. Chapter 12: Managing a Feature Engineering Pipeline in Training and Inference
    1. Technical requirements
    2. Using pipelines to avoid data leaks
    3. Introducing scikit-learn's Pipeline
    4. Implementing a data pipeline in Python
        1. Time series feature engineering in a pipeline
        2. Regression feature engineering in a pipeline
    5. Summary
22. Chapter 13: Deploying Your XGBoost Model
    1. Technical requirements
    2. Training a model using multithreaded and distributed computing with XGBoost
        1. Using XGBoost's multithreaded features
        2. Training with distributed compute

    3. Packaging a model for production deployment
        1. Turning your model into an application with Streamlit
        2. Deploying the application
        3. Creating APIs to call your model
        4. Creating an API for your model using Flask
        5. Using Docker to put your model into a container
    4. Deploying a model using containers
    5. Considerations when deploying a container in the cloud
    6. Servicing your model using the REST API
        1. Monitoring model performance
        2. Determining when to retrain
        3. Retraining your model
    7. Summary
23. Index
    1. Why subscribe?
24. Other Books You May Enjoy
    1. Packt is searching for authors like you
    2. Share Your Thoughts
    3. Download a free PDF copy of this book

Landmarks

1. Cover
2. Table of Contents
3. Index